Focal Loss for Dense Object Detection

Tsung-Yi Lin Priya Goyal Ross Girshick Kaiming He Piotr Dollár
Facebook AI Research (FAIR)

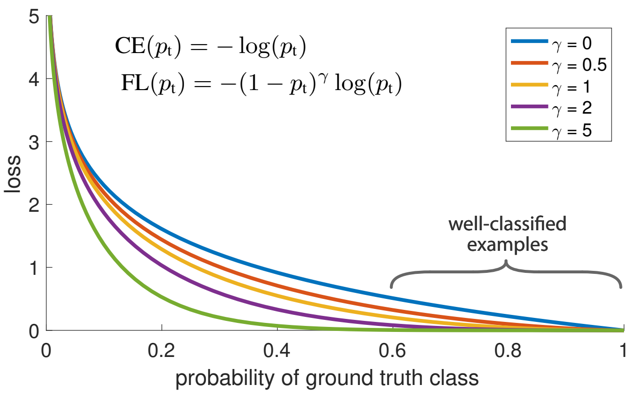


Figure 1. We propose a novel loss we term the *Focal Loss* that adds a factor $(1-p_t)^\gamma$ to the standard cross entropy criterion. Setting $\gamma > 0$ reduces the relative loss for well-classified examples ($p_t > .5$), putting more focus on hard, misclassified examples. As our experiments will demonstrate, the proposed focal loss enables training highly accurate dense object detectors in the presence of vast numbers of easy background examples.

Abstract

The highest accuracy object detectors to date are based on a two-stage approach popularized by R-CNN, where a classifier is applied to a sparse *set of candidate object locations. In contrast, one-stage detectors that are applied over a regular,* dense *sampling of possible object locations have the potential to be faster and simpler, but have trailed the accuracy of two-stage detectors thus far. In this paper, we investigate why this is the case. We discover that the extreme foreground-background class imbalance encountered during training of dense detectors is the central cause. We propose to address this class imbalance by reshaping the standard cross entropy loss such that it down-weights the loss assigned to well-classified examples. Our novel* Focal Loss *focuses training on a sparse set of hard examples and prevents the vast number of easy negatives from overwhelming the detector during training. To evaluate the effectiveness of our loss, we design and train a simple dense detector we call RetinaNet. Our results show that when trained with the focal loss, RetinaNet is able to match the speed of previous one-stage detectors while surpassing the accuracy of all existing state-of-the-art two-stage detectors. Code is at: https://github.com/facebookresearch/Detectron.*

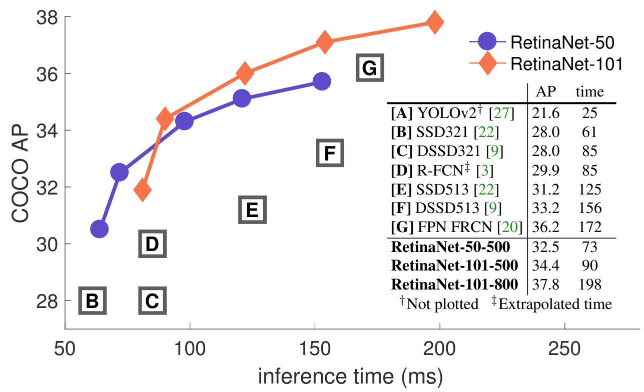


Figure 2. Speed (ms) versus accuracy (AP) on COCO `test-dev`. Enabled by the focal loss, our simple one-stage *RetinaNet* detector outperforms all previous one-stage and two-stage detectors, including the best reported Faster R-CNN [28] system from [20]. We show variants of RetinaNet with ResNet-50-FPN (blue circles) and ResNet-101-FPN (orange diamonds) at five scales (400-800 pixels). Ignoring the low-accuracy regime (AP<25), RetinaNet forms an upper envelope of all current detectors, and an improved variant (not shown) achieves 40.8 AP. Details are given in §5.

1. Introduction

Current state-of-the-art object detectors are based on a two-stage, proposal-driven mechanism. As popularized in the R-CNN framework [11], the first stage generates a *sparse* set of candidate object locations and the second stage classifies each candidate location as one of the foreground classes or as background using a convolutional neural network. Through a sequence of advances [10, 28, 20, 14], this two-stage framework consistently achieves top accuracy on the challenging COCO benchmark [21].

Despite the success of two-stage detectors, a natural question to ask is: could a simple one-stage detector achieve similar accuracy? One stage detectors are applied over a regular, *dense* sampling of object locations, scales, and aspect ratios. Recent work on one-stage detectors, such as YOLO [26, 27] and SSD [22, 9], demonstrates promising results, yielding faster detectors with accuracy within 10-40% relative to state-of-the-art two-stage methods.

This paper pushes the envelop further: we present a one-stage object detector that, for the first time, matches the state-of-the-art COCO AP of more complex two-stage de-

密集目标检测的焦点损失

林致远 普里亚·戈亚尔 罗斯·吉斯克 何恺明 皮奥特·多拉Facebook AI Research (FAIR)

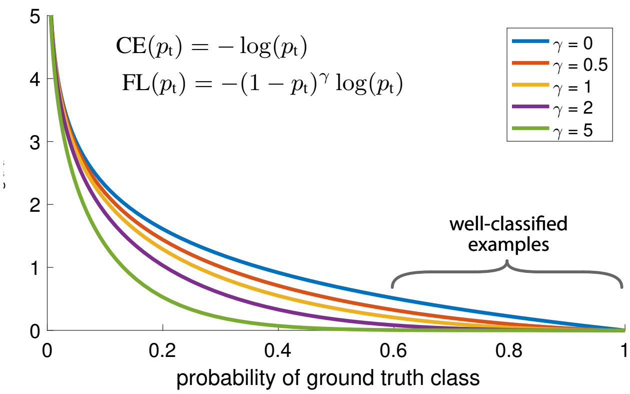
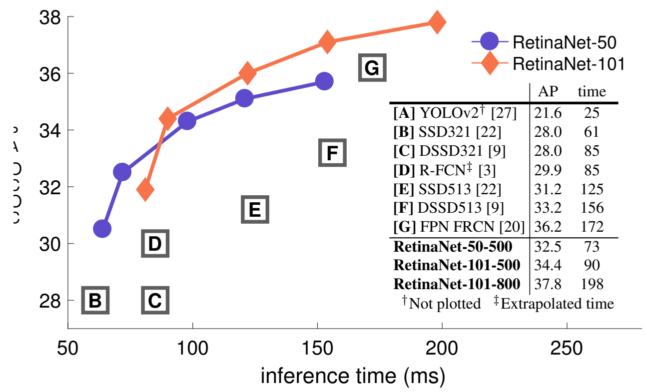


图1. 我们提出了一种新颖的损失函数，称为*Focal Loss*，它在标准交叉熵准则中增加了一个因子$(1-p_t)^{\gamma}$。设定$\gamma>0$会降低分类良好样本（$p_t>.5$）的相对损失，从而更关注难以分类的错误样本。如实验所示，所提出的焦点损失能够在存在大量简单背景样本的情况下，训练出高精度的密集目标检测器。



	AP	time
[A] YOLOv2† [27]	21.6	25
[B] SSD321 [22]	28.0	61
[C] DSSD321 [9]	28.0	85
[D] R-FCN‡ [3]	29.9	85
[E] SSD513 [22]	31.2	125
[F] DSSD513 [9]	33.2	156
[G] FPN FRCN [20]	36.2	172
RetinaNet-50-500	32.5	73
RetinaNet-101-500	34.4	90
RetinaNet-101-800	37.8	198

†Not plotted ‡Extrapolated time

图2. COCO测试开发集上的速度（毫秒）与精度（AP）对比。得益于焦点损失函数，我们简单的单阶段*RetinaNet*检测器超越了所有先前的单阶段和两阶段检测器，包括文献[20]中报告的最佳Faster R-CNN[28]系统。我们展示了RetinaNet在五个尺度（400-800像素）下使用ResNet-50-FPN（蓝色圆圈）和ResNet-101-FPN（橙色菱形）的变体。在忽略低精度区域（AP<25）后，RetinaNet构成了当前所有检测器的性能上限边界，而一个改进变体（未展示）达到了40.8 AP。详细内容见§5。

摘要

The highest accuracy object detectors to date are based on a two-stage approach popularized by R-CNN, where a classifier is applied to a 稀疏 *set of candidate object locations. In contrast, one-stage detectors that are applied over a regular,* 密集 *sampling of possible object locations have the potential to be faster and simpler, but have trailed the accuracy of two-stage detectors thus far. In this paper, we investigate why this is the case. We discover that the extreme foreground-background class imbalance encountered during training of dense detectors is the central cause. We propose to address this class imbalance by reshaping the standard cross entropy loss such that it down-weights the loss assigned to well-classified examples. Our novel* Focal Loss *focuses training on a sparse set of hard examples and prevents the vast number of easy negatives from overwhelming the detector during training. To evaluate the effectiveness of our loss, we design and train a simple dense detector we call RetinaNet. Our results show that when trained with the focal loss, RetinaNet is able to match the speed of previous one-stage detectors while surpassing the accuracy of all existing state-of-the-art two-stage detectors. Code is at:* https://github.com/facebookresearch/Detectron.

1. 引言

当前最先进的物体检测器基于两阶段、提议驱动的机制。正如R-CNN框架[11]所推广的，第一阶段生成一组候选物体位置，第二阶段使用卷积神经网络将每个候选位置分类为前景类别之一或背景。通过一系列改进[10, 28, 20, 14]，这种两阶段框架在具有挑战性的COCO基准测试[21]中始终保持着最高的准确率。

尽管两阶段检测器取得了成功，但一个自然而然的问题是：简单的单阶段检测器能否达到相似的精度？单阶段检测器在物体位置、尺度和宽高比上采用规则且密集的采样方式。近期关于单阶段检测器的研究，如YOLO [26, 27]和SSD [22, 9]，展示了令人鼓舞的成果，这些检测器速度更快，且准确率与最先进的两阶段方法相比差距在10-40%以内。

本文进一步拓展了边界：我们提出了一种单阶段目标检测器，首次在COCO AP指标上达到了与更复杂双阶段检测器相当的最先进水平。

tectors, such as the Feature Pyramid Network (FPN) [20] or Mask R-CNN [14] variants of Faster R-CNN [28]. To achieve this result, we identify class imbalance during training as the main obstacle impeding one-stage detector from achieving state-of-the-art accuracy and propose a new loss function that eliminates this barrier.

Class imbalance is addressed in R-CNN-like detectors by a two-stage cascade and sampling heuristics. The proposal stage (*e.g*., Selective Search [35], EdgeBoxes [39], DeepMask [24, 25], RPN [28]) rapidly narrows down the number of candidate object locations to a small number (*e.g*., 1-2k), filtering out most background samples. In the second classification stage, sampling heuristics, such as a fixed foreground-to-background ratio (1:3), or online hard example mining (OHEM) [31], are performed to maintain a manageable balance between foreground and background.

In contrast, a one-stage detector must process a much larger set of candidate object locations regularly sampled across an image. In practice this often amounts to enumerating ~100k locations that densely cover spatial positions, scales, and aspect ratios. While similar sampling heuristics may also be applied, they are inefficient as the training procedure is still dominated by easily classified background examples. This inefficiency is a classic problem in object detection that is typically addressed via techniques such as bootstrapping [33, 29] or hard example mining [37, 8, 31].

In this paper, we propose a new loss function that acts as a more effective alternative to previous approaches for dealing with class imbalance. The loss function is a dynamically scaled cross entropy loss, where the scaling factor decays to zero as confidence in the correct class increases, see Figure 1. Intuitively, this scaling factor can automatically down-weight the contribution of easy examples during training and rapidly focus the model on hard examples. Experiments show that our proposed *Focal Loss* enables us to train a high-accuracy, one-stage detector that significantly outperforms the alternatives of training with the sampling heuristics or hard example mining, the previous state-of-the-art techniques for training one-stage detectors. Finally, we note that the exact form of the focal loss is not crucial, and we show other instantiations can achieve similar results.

To demonstrate the effectiveness of the proposed focal loss, we design a simple one-stage object detector called *RetinaNet*, named for its dense sampling of object locations in an input image. Its design features an efficient in-network feature pyramid and use of anchor boxes. It draws on a variety of recent ideas from [22, 6, 28, 20]. RetinaNet is efficient and accurate; our best model, based on a ResNet-101-FPN backbone, achieves a COCO `test-dev` AP of 39.1 while running at 5 fps, surpassing the previously best published single-model results from both one and two-stage detectors, see Figure 2.

2. Related Work

Classic Object Detectors: The sliding-window paradigm, in which a classifier is applied on a dense image grid, has a long and rich history. One of the earliest successes is the classic work of LeCun *et al*. who applied convolutional neural networks to handwritten digit recognition [19, 36]. Viola and Jones [37] used boosted object detectors for face detection, leading to widespread adoption of such models. The introduction of HOG [4] and integral channel features [5] gave rise to effective methods for pedestrian detection. DPMs [8] helped extend dense detectors to more general object categories and had top results on PASCAL [7] for many years. While the sliding-window approach was the leading detection paradigm in classic computer vision, with the resurgence of deep learning [18], two-stage detectors, described next, quickly came to dominate object detection.

Two-stage Detectors: The dominant paradigm in modern object detection is based on a two-stage approach. As pioneered in the Selective Search work [35], the first stage generates a sparse set of candidate proposals that should contain all objects while filtering out the majority of negative locations, and the second stage classifies the proposals into foreground classes / background. R-CNN [11] upgraded the second-stage classifier to a convolutional network yielding large gains in accuracy and ushering in the modern era of object detection. R-CNN was improved over the years, both in terms of speed [15, 10] and by using learned object proposals [6, 24, 28]. Region Proposal Networks (RPN) integrated proposal generation with the second-stage classifier into a single convolution network, forming the Faster R-CNN framework [28]. Numerous extensions to this framework have been proposed, *e.g*. [20, 31, 32, 16, 14].

One-stage Detectors: OverFeat [30] was one of the first modern one-stage object detector based on deep networks. More recently SSD [22, 9] and YOLO [26, 27] have renewed interest in one-stage methods. These detectors have been tuned for speed but their accuracy trails that of two-stage methods. SSD has a 10-20% lower AP, while YOLO focuses on an even more extreme speed/accuracy trade-off. See Figure 2. Recent work showed that two-stage detectors can be made fast simply by reducing input image resolution and the number of proposals, but one-stage methods trailed in accuracy even with a larger compute budget [17]. In contrast, the aim of this work is to understand if one-stage detectors can match or surpass the accuracy of two-stage detectors while running at similar or faster speeds.

The design of our RetinaNet detector shares many similarities with previous dense detectors, in particular the concept of 'anchors' introduced by RPN [28] and use of features pyramids as in SSD [22] and FPN [20]. We emphasize that our simple detector achieves top results not based on innovations in network design but due to our novel loss.

例如特征金字塔网络（FPN）[20]或Faster R-CNN [28]的Mask R-CNN [14]变体。为实现这一目标，我们将训练过程中的类别不平衡确定为阻碍单阶段检测器达到最先进精度的主要障碍，并提出一种消除这一障碍的新损失函数。

类不平衡问题在R-CNN类检测器中通过两阶段级联和采样启发式方法得到解决。候选框生成阶段（{v*}如选择性搜索[35]、EdgeBoxes[39]、DeepMask[24,25]、RPN[28]）将候选目标位置数量迅速缩减至较小规模（{v*}如1-2k），从而过滤掉大多数背景样本。在第二阶段的分类任务中，采用固定前景背景比例（1:3）或在线难例挖掘（OHEM）[31]等采样启发式策略，以维持前景与背景样本间的可控平衡。

相比之下，单阶段检测器必须处理图像中定期采样得到的更大规模候选目标位置集合。实际应用中，这通常需要枚举约10万个密集覆盖空间位置、尺度与宽高比的~候选区域。虽然类似的采样启发式方法也可应用，但由于训练过程仍被易于分类的背景样本所主导，这些方法效率低下。这种低效性是目标检测领域的经典难题，通常通过自举法[33, 29]或难例挖掘[37, 8, 31]等技术来解决。

本文提出一种新的损失函数，作为处理类别不平衡问题的更有效替代方案。该损失函数是一种动态缩放的交叉熵损失，其缩放因子随着对正确类别置信度的增加而衰减至零，如图1所示。直观而言，这种缩放因子能自动降低简单样本在训练中的贡献权重，使模型快速聚焦于困难样本。实验表明，我们提出的*Focal Loss*能够训练出高精度的单阶段检测器，其性能显著优于采用采样启发式方法或困难样本挖掘等先前训练单阶段检测器的前沿技术。最后需要说明，焦点损失的具体形式并非关键因素，我们展示了其他实现方案也能取得相似效果。

为验证所提出的焦点损失函数的有效性，我们设计了一个名为*RetinaNet*的简单单阶段目标检测器，其名称源于对输入图像中目标位置的密集采样。该设计采用了高效的网络内特征金字塔结构并运用了锚框技术，融合了[22, 6, 28, 20]等多项近期研究成果。RetinaNet兼具高效性与准确性；我们基于ResNet-101-FPN主干网络的最佳模型在COCO test-dev数据集上实现了39.1的AP值，同时以5帧/秒的速度运行，超越了此前已发表的最佳单模型结果（包括单阶段与双阶段检测器），详见图2。

2. 相关工作

经典目标检测器：滑动窗口范式，即在密集图像网格上应用分类器，拥有悠久而丰富的历史。最早的成果之一是LeCun *et al*的经典工作，他将卷积神经网络应用于手写数字识别[19, 36]。Viola和Jones[37]使用增强目标检测器进行人脸检测，推动了此类模型的广泛应用。HOG[4]和积分通道特征[5]的引入催生了有效的行人检测方法。DPM[8]帮助将密集检测器扩展到更通用的目标类别，并在PASCAL[7]数据集上保持多年领先结果。虽然滑动窗口方法是经典计算机视觉中的主流检测范式，但随着深度学习[18]的复兴，接下来描述的两阶段检测器迅速主导了目标检测领域。

两阶段检测器：现代目标检测的主流范式基于两阶段方法。如选择性搜索工作[35]所开创，第一阶段生成稀疏的候选建议区域集合，这些区域应包含所有目标，同时过滤掉大部分负样本位置；第二阶段将建议区域分类为前景类别/背景。R-CNN[11]将第二阶段分类器升级为卷积网络，显著提升了准确率，并开启了目标检测的新时代。R-CNN历经多年改进，在速度[15, 10]和学习型目标建议区域[6, 24, 28]方面均有提升。区域建议网络（RPN）将建议生成与第二阶段分类器整合到单一卷积网络中，形成了Faster R-CNN框架[28]。该框架已衍生出众多扩展，*e.g*。[20, 31, 32, 16, 14]。

单阶段检测器：OverFeat [30] 是基于深度网络的首批现代单阶段目标检测器之一。近年来，SSD [22, 9] 和 YOLO [26, 27] 重新激发了人们对单阶段方法的兴趣。这些检测器已针对速度进行优化，但其精度仍落后于两阶段方法。SSD 的平均精度（AP）低 10-20%，而 YOLO 则侧重于更极端的速度/精度权衡。参见图 2。近期研究表明，通过降低输入图像分辨率和候选框数量，两阶段检测器可轻松实现加速，但即使增加计算资源，单阶段方法的精度仍存在差距 [17]。相比之下，本研究旨在探讨单阶段检测器能否在保持相当或更快速度的同时，达到甚至超越两阶段检测器的精度。

我们的RetinaNet检测器设计与之前的密集检测器有许多相似之处，特别是借鉴了RPN [28]引入的“锚点”概念，以及采用了SSD [22]和FPN [20]中的特征金字塔结构。需要强调的是，我们简单的检测器之所以能达到顶尖效果，并非源于网络设计的创新，而是得益于我们新颖的损失函数。

Class Imbalance: Both classic one-stage object detection methods, like boosted detectors [37, 5] and DPMs [8], and more recent methods, like SSD [22], face a large class imbalance during training. These detectors evaluate 10^4-10^5 candidate locations per image but only a few locations contain objects. This imbalance causes two problems: (1) training is inefficient as most locations are easy negatives that contribute no useful learning signal; (2) en masse, the easy negatives can overwhelm training and lead to degenerate models. A common solution is to perform some form of hard negative mining [33, 37, 8, 31, 22] that samples hard examples during training or more complex sampling/reweighing schemes [2]. In contrast, we show that our proposed focal loss naturally handles the class imbalance faced by a one-stage detector and allows us to efficiently train on all examples without sampling and without easy negatives overwhelming the loss and computed gradients.

Robust Estimation: There has been much interest in designing robust loss functions (*e.g.*, Huber loss [13]) that reduce the contribution of *outliers* by down-weighting the loss of examples with large errors (hard examples). In contrast, rather than addressing outliers, our focal loss is designed to address class imbalance by down-weighting *inliers* (easy examples) such that their contribution to the total loss is small even if their number is large. In other words, the focal loss performs the *opposite* role of a robust loss: it focuses training on a sparse set of hard examples.

3. Focal Loss

The *Focal Loss* is designed to address the one-stage object detection scenario in which there is an extreme imbalance between foreground and background classes during training (*e.g.*, 1:1000). We introduce the focal loss starting from the cross entropy (CE) loss for binary classification[1]:

$$\text{CE}(p, y) = \begin{cases} -\log(p) & \text{if } y = 1 \\ -\log(1-p) & \text{otherwise.} \end{cases} \tag{1}$$

In the above $y \in \{\pm 1\}$ specifies the ground-truth class and $p \in [0, 1]$ is the model's estimated probability for the class with label $y = 1$. For notational convenience, we define p_t:

$$p_\text{t} = \begin{cases} p & \text{if } y = 1 \\ 1-p & \text{otherwise,} \end{cases} \tag{2}$$

and rewrite $\text{CE}(p, y) = \text{CE}(p_\text{t}) = -\log(p_\text{t})$.

The CE loss can be seen as the blue (top) curve in Figure 1. One notable property of this loss, which can be easily seen in its plot, is that even examples that are easily classified ($p_\text{t} \gg .5$) incur a loss with non-trivial magnitude. When summed over a large number of easy examples, these small loss values can overwhelm the rare class.

[1]Extending the focal loss to the multi-class case is straightforward and works well; for simplicity we focus on the binary loss in this work.

3.1. Balanced Cross Entropy

A common method for addressing class imbalance is to introduce a weighting factor $\alpha \in [0, 1]$ for class 1 and $1 - \alpha$ for class -1. In practice α may be set by inverse class frequency or treated as a hyperparameter to set by cross validation. For notational convenience, we define α_t analogously to how we defined p_t. We write the α-balanced CE loss as:

$$\text{CE}(p_\text{t}) = -\alpha_\text{t} \log(p_\text{t}). \tag{3}$$

This loss is a simple extension to CE that we consider as an experimental baseline for our proposed focal loss.

3.2. Focal Loss Definition

As our experiments will show, the large class imbalance encountered during training of dense detectors overwhelms the cross entropy loss. Easily classified negatives comprise the majority of the loss and dominate the gradient. While α balances the importance of positive/negative examples, it does not differentiate between easy/hard examples. Instead, we propose to reshape the loss function to down-weight easy examples and thus focus training on hard negatives.

More formally, we propose to add a modulating factor $(1 - p_\text{t})^\gamma$ to the cross entropy loss, with tunable *focusing* parameter $\gamma \geq 0$. We define the focal loss as:

$$\text{FL}(p_\text{t}) = -(1 - p_\text{t})^\gamma \log(p_\text{t}). \tag{4}$$

The focal loss is visualized for several values of $\gamma \in [0, 5]$ in Figure 1. We note two properties of the focal loss. (1) When an example is misclassified and p_t is small, the modulating factor is near 1 and the loss is unaffected. As $p_\text{t} \rightarrow 1$, the factor goes to 0 and the loss for well-classified examples is down-weighted. (2) The focusing parameter γ smoothly adjusts the rate at which easy examples are down-weighted. When $\gamma = 0$, FL is equivalent to CE, and as γ is increased the effect of the modulating factor is likewise increased (we found $\gamma = 2$ to work best in our experiments).

Intuitively, the modulating factor reduces the loss contribution from easy examples and extends the range in which an example receives low loss. For instance, with $\gamma = 2$, an example classified with $p_\text{t} = 0.9$ would have $100\times$ lower loss compared with CE and with $p_\text{t} \approx 0.968$ it would have $1000\times$ lower loss. This in turn increases the importance of correcting misclassified examples (whose loss is scaled down by at most $4\times$ for $p_\text{t} \leq .5$ and $\gamma = 2$).

In practice we use an α-balanced variant of the focal loss:

$$\text{FL}(p_\text{t}) = -\alpha_\text{t}(1 - p_\text{t})^\gamma \log(p_\text{t}). \tag{5}$$

We adopt this form in our experiments as it yields slightly improved accuracy over the non-α-balanced form. Finally, we note that the implementation of the loss layer combines the sigmoid operation for computing p with the loss computation, resulting in greater numerical stability.

类别不平衡：无论是经典的单阶段目标检测方法，如增强检测器[37, 5]和DPMs[8]，还是更现代的方法，如SSD[22]，在训练过程中都面临严重的类别不平衡问题。这些检测器每张图像评估10^4到10^5个候选位置，但只有少数位置包含目标。这种不平衡会导致两个问题：(1) 训练效率低下，因为大多数位置是容易分类的负样本，无法提供有效的学习信号；(2) 大量容易分类的负样本会主导训练过程，导致模型退化。常见的解决方案是采用某种形式的难负样本挖掘[33, 37, 8, 31, 22]，在训练中采样困难样本，或使用更复杂的采样/重加权策略[2]。相比之下，我们提出的焦点损失函数能自然处理单阶段检测器面临的类别不平衡问题，使我们能够高效地利用所有样本进行训练，无需采样，且避免容易分类的负样本主导损失函数和梯度计算。

鲁棒估计：设计鲁棒损失函数（*e.g*，如Huber损失[13]）已引起广泛关注，这类函数通过降低误差较大样本（困难样本）的损失权重，从而减少*outliers*的影响。与之相反，我们的焦点损失并非针对异常值，而是旨在通过降低*inliers* (简单样本)的权重来解决类别不平衡问题，使得即使简单样本数量众多，它们对总损失的贡献也很小。换言之，焦点损失承担了与鲁棒损失*opposite*类似的角色：它将训练聚焦于少量困难样本构成的稀疏集合上。

3. 焦点损失

*Focal Loss*旨在解决单阶段目标检测场景中，训练时前景与背景类别间存在极端不平衡的问题（*e.g*，比例约为1:1000）。我们从二元分类的交叉熵（CE）损失出发，引入焦点损失[1]：

$$\mathrm{CE}(p,y) = \begin{cases} -\log(p) & \text{if } y=1 \\ -\log(1-p) & \text{otherwise.} \end{cases} \tag{1}$$

在上述公式中，$y \in \{\pm 1\}$ 指定了真实类别，而 $p \in [0, 1]$ 是模型对标签为 $y=1$ 的类别的估计概率。为便于标记，我们定义 p_{t}:

$$p_{\mathrm{t}} = \begin{cases} p & \text{if } y=1 \\ 1-p & \text{otherwise,} \end{cases} \tag{2}$$

并重写$\mathrm{CE}(p,y) = \mathrm{CE}(p_{\mathrm{t}}) = -\log(p_{\mathrm{t}})$。

CE损失可以看作是图1中的蓝色（顶部）曲线。该损失的一个显著特性（从其曲线图中可以轻易看出）是：即使是容易分类的样本（$p_{\mathrm{t}} \gg .5$）也会产生不可忽略的损失值。当大量简单样本的损失叠加时，这些微小的损失值可能会淹没稀有类别的信号。

[1]Extending the focal loss to the multi-class case is straightforward and works well; for simplicity we focus on the binary loss in this work.

3.1. 平衡交叉熵

处理类别不平衡的一种常见方法是，为类别1引入一个权重因子$\alpha \in [0, 1]$，为类别-1引入权重$1-\alpha$。在实践中，α可以通过逆类别频率设定，或作为超参数通过交叉验证来设置。为记法方便，我们类似定义α_{t}的方式定义p_{t}。我们将α平衡的CE损失写作：

$$\mathrm{CE}(p_{\mathrm{t}}) = -\alpha_{\mathrm{t}} \log(p_{\mathrm{t}}). \tag{3}$$

该损失是交叉熵的一个简单扩展，我们将其视为我们提出的焦点损失的实验基线。

3.2. 焦点损失定义

正如我们的实验将展示的，密集检测器训练中遇到的大规模类别不平衡问题会淹没交叉熵损失。易于分类的负样本构成了损失的主要部分并主导了梯度。虽然{v*}平衡了正负样本的重要性，但并未区分难易样本。为此，我们提出重构损失函数，以降低简单样本的权重，从而使训练更专注于困难负样本。

更正式地说，我们建议在交叉熵损失中加入一个可调节因子 $(1-p_{\mathrm{t}})^{\gamma}$，其中可调参数 *focusing* $\gamma \geq 0$。我们将焦点损失定义为：

$$\mathrm{FL}(p_{\mathrm{t}}) = -(1-p_{\mathrm{t}})^{\gamma} \log(p_{\mathrm{t}}). \tag{4}$$

焦点损失在图1中针对几个$\gamma \in [0, 5]$的值进行了可视化。我们注意到焦点损失的两个特性。(1) 当一个样本被错误分类且p_{t}较小时，调制因子接近1，损失不受影响。随着$p_{\mathrm{t}} \rightarrow$趋近于1，该因子趋于0，分类良好样本的损失被降低权重。(2) 聚焦参数γ平滑地调整简单样本被降低权重的速率。当$\gamma=$为0时，FL等价于CE；随着γ增大，调制因子的影响也相应增强（我们在实验中发现$\gamma=$为2时效果最佳）。

直观地说，调制因子降低了简单样本的损失贡献，并扩大了样本获得低损失的范围。例如，当 $\gamma = 2$ 时，一个被分类为 $p_{\mathrm{t}} = 0.9$ 的样本将比交叉熵损失降低 100×，而当 $p_{\mathrm{t}} \approx 0.968$ 时，损失将降低 1000×。这反过来增加了纠正错误分类样本的重要性（对于 $p_{\mathrm{t}} \leq .5$ 和 $\gamma = 2$ 的情况，其损失最多被缩小 4×）。

在实践中，我们使用一种α平衡的焦点损失变体：

$$\mathrm{FL}(p_{\mathrm{t}}) = -\alpha_{\mathrm{t}}(1-p_{\mathrm{t}})^{\gamma} \log(p_{\mathrm{t}}). \tag{5}$$

我们在实验中采用这种形式，因为它比非α平衡形式能带来略微提升的准确度。最后需要说明的是，损失层的实现将计算p的sigmoid操作与损失计算相结合，从而获得了更好的数值稳定性。

While in our main experimental results we use the focal loss definition above, its precise form is not crucial. In the appendix we consider other instantiations of the focal loss and demonstrate that these can be equally effective.

3.3. Class Imbalance and Model Initialization

Binary classification models are by default initialized to have equal probability of outputting either $y = -1$ or 1. Under such an initialization, in the presence of class imbalance, the loss due to the frequent class can dominate total loss and cause instability in early training. To counter this, we introduce the concept of a 'prior' for the value of p estimated by the model for the rare class (foreground) *at the start of training*. We denote the prior by π and set it so that the model's estimated p for examples of the rare class is low, *e.g.* 0.01. We note that this is a change in model initialization (see §4.1) and *not* of the loss function. We found this to improve training stability for both the cross entropy and focal loss in the case of heavy class imbalance.

3.4. Class Imbalance and Two-stage Detectors

Two-stage detectors are often trained with the cross entropy loss without use of α-balancing or our proposed loss. Instead, they address class imbalance through two mechanisms: (1) a two-stage cascade and (2) biased minibatch sampling. The first cascade stage is an object proposal mechanism [35, 24, 28] that reduces the nearly infinite set of possible object locations down to one or two thousand. Importantly, the selected proposals are not random, but are likely to correspond to true object locations, which removes the vast majority of easy negatives. When training the second stage, biased sampling is typically used to construct minibatches that contain, for instance, a 1:3 ratio of positive to negative examples. This ratio is like an implicit α-balancing factor that is implemented via sampling. Our proposed focal loss is designed to address these mechanisms in a one-stage detection system directly via the loss function.

4. RetinaNet Detector

RetinaNet is a single, unified network composed of a *backbone* network and two task-specific *subnetworks*. The backbone is responsible for computing a convolutional feature map over an entire input image and is an off-the-self convolutional network. The first subnet performs convolutional object classification on the backbone's output; the second subnet performs convolutional bounding box regression. The two subnetworks feature a simple design that we propose specifically for one-stage, dense detection, see Figure 3. While there are many possible choices for the details of these components, most design parameters are not particularly sensitive to exact values as shown in the experiments. We describe each component of RetinaNet next.

Feature Pyramid Network Backbone: We adopt the Feature Pyramid Network (FPN) from [20] as the backbone network for RetinaNet. In brief, FPN augments a standard convolutional network with a top-down pathway and lateral connections so the network efficiently constructs a rich, multi-scale feature pyramid from a single resolution input image, see Figure 3(a)-(b). Each level of the pyramid can be used for detecting objects at a different scale. FPN improves multi-scale predictions from fully convolutional networks (FCN) [23], as shown by its gains for RPN [28] and DeepMask-style proposals [24], as well at two-stage detectors such as Fast R-CNN [10] or Mask R-CNN [14].

Following [20], we build FPN on top of the ResNet architecture [16]. We construct a pyramid with levels P_3 through P_7, where l indicates pyramid level (P_l has resolution 2^l lower than the input). As in [20] all pyramid levels have $C = 256$ channels. Details of the pyramid generally follow [20] with a few modest differences.[2] While many design choices are not crucial, we emphasize the use of the FPN backbone is; preliminary experiments using features from only the final ResNet layer yielded low AP.

Anchors: We use translation-invariant anchor boxes similar to those in the RPN variant in [20]. The anchors have areas of 32^2 to 512^2 on pyramid levels P_3 to P_7, respectively. As in [20], at each pyramid level we use anchors at three aspect ratios {1:2, 1:1, 2:1}. For denser scale coverage than in [20], at each level we add anchors of sizes {2^0, $2^{1/3}$, $2^{2/3}$} of the original set of 3 aspect ratio anchors. This improve AP in our setting. In total there are $A = 9$ anchors per level and across levels they cover the scale range 32 - 813 pixels with respect to the network's input image.

Each anchor is assigned a length K one-hot vector of classification targets, where K is the number of object classes, and a 4-vector of box regression targets. We use the assignment rule from RPN [28] but modified for multiclass detection and with adjusted thresholds. Specifically, anchors are assigned to ground-truth object boxes using an intersection-over-union (IoU) threshold of 0.5; and to background if their IoU is in [0, 0.4). As each anchor is assigned to at most one object box, we set the corresponding entry in its length K label vector to 1 and all other entries to 0. If an anchor is unassigned, which may happen with overlap in [0.4, 0.5), it is ignored during training. Box regression targets are computed as the offset between each anchor and its assigned object box, or omitted if there is no assignment.

[2] RetinaNet uses feature pyramid levels P_3 to P_7, where P_3 to P_5 are computed from the output of the corresponding ResNet residual stage (C_3 through C_5) using top-down and lateral connections just as in [20], P_6 is obtained via a 3×3 stride-2 conv on C_5, and P_7 is computed by applying ReLU followed by a 3×3 stride-2 conv on P_6. This differs slightly from [20]: (1) we don't use the high-resolution pyramid level P_2 for computational reasons, (2) P_6 is computed by strided convolution instead of downsampling, and (3) we include P_7 to improve large object detection. These minor modifications improve speed while maintaining accuracy.

虽然我们在主要实验结果中使用了上述焦点损失的定义，但其精确形式并非关键。在附录中，我们考虑了焦点损失的其他实现方式，并证明这些方法同样有效。

3.3. 类别不平衡与模型初始化

二元分类模型默认初始化为输出$y = -1$或1的概率相等。在这种初始化下，当存在类别不平衡时，频繁类别造成的损失可能主导总损失，导致早期训练不稳定。为应对此问题，我们引入“先验”概念，用于表示模型对稀有类别（前景）*at thestart of training*所估计的p值。我们将该先验记为π，并将其设置为使模型对稀有类别样本的估计p较低，即*e.g*。0.01。需注意这是模型初始化方式的调整（参见§4.1节）以及损失函数的*not*。我们发现这一改进能显著提升在严重类别不平衡情况下，交叉熵损失和焦点损失的训练稳定性。

3.4. 类别不平衡与两阶段检测器

两阶段检测器通常使用交叉熵损失进行训练，既不采用α平衡方法，也不使用我们提出的损失函数。相反，它们通过两种机制处理类别不平衡问题：（1）两阶段级联结构；（2）有偏的小批量采样。第一级级联是目标候选框生成机制[35, 24, 28]，它将近乎无限的可能目标位置缩减至一千到两千个。关键的是，所选候选框并非随机产生，而很可能对应真实目标位置，从而消除了大量简单负样本。在训练第二阶段时，通常采用有偏采样构建小批量数据，例如使正负样本比例维持在1:3。这种采样方式如同通过隐式α平衡因子实现类别平衡。我们提出的焦点损失函数，旨在通过损失函数本身直接在单阶段检测系统中实现上述机制的效果。

4. RetinaNet 检测器

RetinaNet是一个由*backbone*网络和两个任务特定的*subnetworks*组成的单一统一网络。主干网络负责在整个输入图像上计算卷积特征图，是一个现成的卷积网络。第一个子网在主干网络的输出上执行卷积目标分类；第二个子网执行卷积边界框回归。这两个子网络采用我们专门为单阶段密集检测提出的简单设计，如图3所示。尽管这些组件的细节有许多可能的选择，但大多数设计参数对具体数值并不特别敏感，如实验所示。接下来我们将描述RetinaNet的每个组件。

特征金字塔网络主干：我们采用[20]中的特征金字塔网络（FPN）作为RetinaNet的主干网络。简而言之，FPN通过自上而下的路径和横向连接增强标准卷积网络，使得网络能够从单一分辨率的输入图像高效构建丰富的多尺度特征金字塔，见图3(a)-(b)。金字塔的每一层可用于检测不同尺度的目标。FPN提升了全卷积网络（FCN）[23]的多尺度预测能力，其在RPN[28]和DeepMask式提案网络[24]中的性能提升已得到验证，同时也优化了两阶段检测器如Fast R-CNN[10]或Mask R-CNN[14]的表现。

遵循[20]的方法，我们在ResNet架构[16]的基础上构建了FPN。我们构建了一个包含P_3到P_7层的金字塔，其中l表示金字塔层级（P_l的分辨率比输入低$2^{\wedge l}$倍）。如[20]所述，所有金字塔层级均具有C =个256通道。金字塔的细节基本遵循[20]，仅有少量细微调整。[2]尽管许多设计选择并非关键，但我们强调使用FPN骨干网络是至关重要的；仅使用最终ResNet层特征的初步实验导致了较低的AP值。

锚点：我们采用与[20]中RPN变体类似的平移不变锚框。锚框在金字塔层级P_3至P_7上的面积分别为32^2至512^2。如[20]所述，在每个金字塔层级上我们使用三种宽高比$\{$1:2,、1:1、2:1$\}$的锚框。为了获得比[20]更密集的尺度覆盖，我们在每个层级额外添加尺寸为原始3种宽高比锚框$\{2^0$、$2^{1/3}$、$2^{2/3}\}$倍的锚框。这一改进在我们的设置中提升了AP指标。每个层级共有A =9个锚框，所有层级的锚框共同覆盖网络输入图像中32至813像素的尺度范围。

每个锚点被分配一个长度为K的单热向量作为分类目标，其中K是物体类别数量，以及一个4维向量作为边界框回归目标。我们采用RPN[28]中的分配规则，但针对多类别检测进行了修改并调整了阈值。具体而言，当锚点与真实物体框的交并比（IoU）达到0.5时，将其分配给对应物体框；若交并比处于[0, 0.4)区间则归为背景。由于每个锚点最多被分配到一个物体框，我们将其长度为K的标签向量中对应位置设为1，其余位置设为0。若锚点未被分配（可能发生在交并比处于[0.4, 0.5)的重叠情况），则在训练过程中被忽略。边界框回归目标计算为每个锚点与其分配物体框之间的偏移量，若无分配则省略此项。

[2] RetinaNet uses feature pyramid levels P_3 to P_7, where P_3 to P_5 are computed from the output of the corresponding ResNet residual stage (C_3 through C_5) using top-down and lateral connections just as in [20], P_6 is obtained via a 3×3 stride-2 conv on C_5, and P_7 is computed by applying ReLU followed by a 3×3 stride-2 conv on P_6. This differs slightly from [20]: (1) we don't use the high-resolution pyramid level P_2 for computational reasons, (2) P_6 is computed by strided convolution instead of downsampling, and (3) we include P_7 to improve large object detection. These minor modifications improve speed while maintaining accuracy.

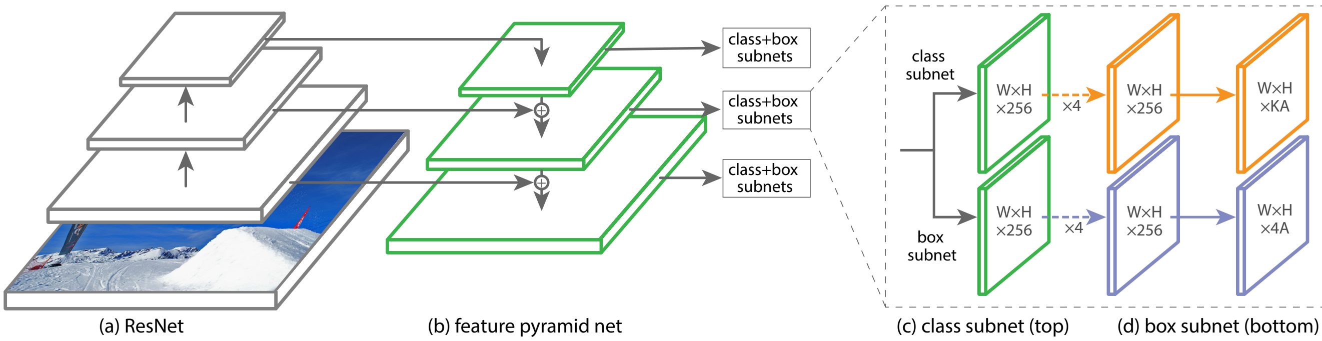


Figure 3. The one-stage **RetinaNet** network architecture uses a Feature Pyramid Network (FPN) [20] backbone on top of a feedforward ResNet architecture [16] (a) to generate a rich, multi-scale convolutional feature pyramid (b). To this backbone RetinaNet attaches two subnetworks, one for classifying anchor boxes (c) and one for regressing from anchor boxes to ground-truth object boxes (d). The network design is intentionally simple, which enables this work to focus on a novel focal loss function that eliminates the accuracy gap between our one-stage detector and state-of-the-art two-stage detectors like Faster R-CNN with FPN [20] while running at faster speeds.

Classification Subnet: The classification subnet predicts the probability of object presence at each spatial position for each of the A anchors and K object classes. This subnet is a small FCN attached to each FPN level; parameters of this subnet are shared across all pyramid levels. Its design is simple. Taking an input feature map with C channels from a given pyramid level, the subnet applies four 3×3 conv layers, each with C filters and each followed by ReLU activations, followed by a 3×3 conv layer with KA filters. Finally sigmoid activations are attached to output the KA binary predictions per spatial location, see Figure 3 (c). We use $C = 256$ and $A = 9$ in most experiments.

In contrast to RPN [28], our object classification subnet is deeper, uses only 3×3 convs, and does not share parameters with the box regression subnet (described next). We found these higher-level design decisions to be more important than specific values of hyperparameters.

Box Regression Subnet: In parallel with the object classification subnet, we attach another small FCN to each pyramid level for the purpose of regressing the offset from each anchor box to a nearby ground-truth object, if one exists. The design of the box regression subnet is identical to the classification subnet except that it terminates in $4A$ linear outputs per spatial location, see Figure 3 (d). For each of the A anchors per spatial location, these 4 outputs predict the relative offset between the anchor and the ground-truth box (we use the standard box parameterization from R-CNN [11]). We note that unlike most recent work, we use a class-agnostic bounding box regressor which uses fewer parameters and we found to be equally effective. The object classification subnet and the box regression subnet, though sharing a common structure, use separate parameters.

4.1. Inference and Training

Inference: RetinaNet forms a single FCN comprised of a ResNet-FPN backbone, a classification subnet, and a box regression subnet, see Figure 3. As such, inference involves simply forwarding an image through the network. To improve speed, we only decode box predictions from at most 1k top-scoring predictions per FPN level, after thresholding detector confidence at 0.05. The top predictions from all levels are merged and non-maximum suppression with a threshold of 0.5 is applied to yield the final detections.

Focal Loss: We use the focal loss introduced in this work as the loss on the output of the classification subnet. As we will show in §5, we find that $\gamma = 2$ works well in practice and the RetinaNet is relatively robust to $\gamma \in [0.5, 5]$. We emphasize that when training RetinaNet, the focal loss is applied to *all* ~100k anchors in each sampled image. This stands in contrast to common practice of using heuristic sampling (RPN) or hard example mining (OHEM, SSD) to select a small set of anchors (*e.g.*, 256) for each minibatch. The total focal loss of an image is computed as the sum of the focal loss over all ~100k anchors, *normalized by the number of anchors assigned to a ground-truth box*. We perform the normalization by the number of assigned anchors, not total anchors, since the vast majority of anchors are easy negatives and receive negligible loss values under the focal loss. Finally we note that α, the weight assigned to the rare class, also has a stable range, but it interacts with γ making it necessary to select the two together (see Tables 1a and 1b). In general α should be decreased slightly as γ is increased (for $\gamma = 2$, $\alpha = 0.25$ works best).

Initialization: We experiment with ResNet-50-FPN and ResNet-101-FPN backbones [20]. The base ResNet-50 and ResNet-101 models are pre-trained on ImageNet1k; we use the models released by [16]. New layers added for FPN are initialized as in [20]. All new conv layers except the final one in the RetinaNet subnets are initialized with bias $b = 0$ and a Gaussian weight fill with $\sigma = 0.01$. For the final conv layer of the classification subnet, we set the bias initialization to $b = -\log((1 - \pi)/\pi)$, where π specifies that at

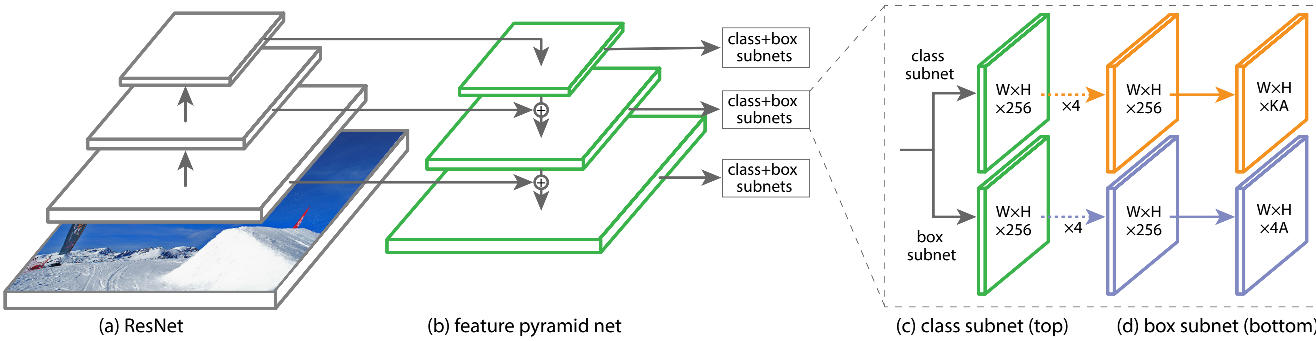


图3. 单阶段RetinaNet网络架构在前馈ResNet架构[16]（a）之上使用特征金字塔网络（FPN）[20]骨干网络（b）生成丰富的多尺度卷积特征金字塔。RetinaNet在此骨干网络上连接两个子网络：一个用于分类锚框（c），另一个用于将锚框回归到真实目标框（d）。该网络设计刻意保持简洁，使本研究能聚焦于一种新颖的焦点损失函数——该函数在保持更快运行速度的同时，消除了我们的单阶段检测器与最先进的双阶段检测器（如带FPN的Faster R-CNN[20]）之间的精度差距。

分类子网：分类子网针对每个空间位置上每个A锚点和K个物体类别，预测物体存在的概率。该子网是一个附加在每个FPN层级的小型全卷积网络（FCN），其参数在所有金字塔层级间共享。其设计十分简洁：对于来自特定金字塔层级、具有C个通道的输入特征图，该子网先应用四个3$\times$3卷积层（每层包含C个滤波器且均后接ReLU激活函数），再连接一个具有KA个滤波器的3$\times$3卷积层，最后通过Sigmoid激活函数输出每个空间位置对应的KA个二元预测结果，详见图3(c)。在大多数实验中，我们设定$C=$为256，$A=$为9。

与RPN[28]不同，我们的物体分类子网络更深，仅使用3$\times$3卷积，且不与边界框回归子网络（将在下文描述）共享参数。我们发现这些更高层次的设计决策比超参数的具体取值更为重要。

边界框回归子网：与物体分类子网并行，我们在每个金字塔层级附加另一个小型全卷积网络，用于回归每个锚框到邻近真实标注物体的偏移量（若存在）。该回归子网的设计与分类子网完全相同，区别在于其在每个空间位置输出4个线性预测值（见图3(d)）。对于每个空间位置的多个锚框，这4个输出值预测锚框与真实标注框之间的相对偏移（采用R-CNN[11]的标准边界框参数化方法）。需要指出的是，与近期多数研究不同，我们采用类别无关的边界框回归器，其参数量更少且经实验验证具有同等有效性。物体分类子网与边界框回归子网虽共享相同网络结构，但使用相互独立的参数。

4.1. 推理与训练

推理：RetinaNet构建了一个单一的全卷积网络，该网络由ResNet-FPN骨干网络、分类子网络和边界框回归子网，如图3所示。因此，推理过程仅涉及将图像前向传播通过网络。为了提高速度，我们在每个FPN层级上，将检测器置信度阈值设为0.05后，仅从最多1k个得分最高的预测中解码边界框预测。所有层级的顶部预测会被合并，并应用阈值为0.5的非极大值抑制，以生成最终检测结果。

Focal Loss：我们采用本工作中提出的焦点损失作为分类子网输出的损失函数。如我们在§5中所示，我们发现$\gamma=$2在实践中效果良好，且RetinaNet对$\gamma\in$[0.5,5]相对稳健。需要强调的是，在训练RetinaNet时，焦点损失会应用于每张采样图像中的*all* ~100k个锚点。这与常见的启发式采样（RPN）或难例挖掘（OHEM、SSD）方法形成对比——这些方法通常为每个小批量选择少量锚点（*e.g*，256）。图像的总焦点损失计算为所有~100k个锚点的焦点损失之和，即*normalized by the number of anchors assigned to a ground-truth box*。我们通过已分配锚点的数量（而非锚点总数）进行归一化，因为绝大多数锚点是简单负样本，在焦点损失下产生的损失值可忽略不计。最后我们注意到，分配给稀有类别的权重α也有一个稳定范围，但它与γ相互影响，因此需要同时选择两者（见表1a和1b）。一般而言，当γ增加时，α应略微降低（对于$\gamma=2$，$\alpha=0.25$效果最佳）。

初始化：我们使用ResNet-50-FPN和ResNet-101-FPN主干网络进行实验[20]。基础的ResNet-50和ResNet-101模型在ImageNet1k上进行了预训练；我们采用[16]发布的模型。为FPN新增的层按照[20]的方法初始化。除RetinaNet子网中最后一层外，所有新增卷积层均以偏置$b=$0和高斯权重填充（标准差为$\sigma=$0.01）进行初始化。对于分类子网的最终卷积层，我们将偏置初始值设为$b=-\log((1-\pi)/\pi)$，其中π表示在

α	AP	AP_{50}	AP_{75}
.10	0.0	0.0	0.0
.25	10.8	16.0	11.7
.50	30.2	46.7	32.8
.75	31.1	49.4	33.0
.90	30.8	49.7	32.3
.99	28.7	47.4	29.9
.999	25.1	41.7	26.1

(a) **Varying α for CE loss** ($\gamma = 0$)

γ	α	AP	AP_{50}	AP_{75}
0	.75	31.1	49.4	33.0
0.1	.75	31.4	49.9	33.1
0.2	.75	31.9	50.7	33.4
0.5	.50	32.9	51.7	35.2
1.0	.25	33.7	52.0	36.2
2.0	.25	**34.0**	**52.5**	**36.5**
5.0	.25	32.2	49.6	34.8

(b) **Varying γ for FL** (w. optimal α)

#sc	#ar	AP	AP_{50}	AP_{75}
1	1	30.3	49.0	31.8
2	1	31.9	50.0	34.0
3	1	31.8	49.4	33.7
1	3	32.4	52.3	33.9
2	3	**34.2**	**53.1**	**36.5**
3	3	34.0	52.5	**36.5**
4	3	33.8	52.1	36.2

(c) **Varying anchor scales and aspects**

method	batch size	nms thr	AP	AP_{50}	AP_{75}
OHEM	128	.7	31.1	47.2	33.2
OHEM	256	.7	31.8	48.8	33.9
OHEM	512	.7	30.6	47.0	32.6
OHEM	128	.5	32.8	50.3	35.1
OHEM	256	.5	31.0	47.4	33.0
OHEM	512	.5	27.6	42.0	29.2
OHEM 1:3	128	.5	31.1	47.2	33.2
OHEM 1:3	256	.5	28.3	42.4	30.3
OHEM 1:3	512	.5	24.0	35.5	25.8
FL	n/a	n/a	**36.0**	**54.9**	**38.7**

(d) **FL *vs.* OHEM** baselines (with ResNet-101-FPN)

depth	scale	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	time
50	400	30.5	47.8	32.7	11.2	33.8	46.1	64
50	500	32.5	50.9	34.8	13.9	35.8	46.7	72
50	600	34.3	53.2	36.9	16.2	37.4	47.4	98
50	700	35.1	54.2	37.7	18.0	39.3	46.4	121
50	800	35.7	55.0	38.5	18.9	38.9	46.3	153
101	400	31.9	49.5	34.1	11.6	35.8	48.5	81
101	500	34.4	53.1	36.8	14.7	38.5	49.1	90
101	600	36.0	55.2	38.7	17.4	39.6	49.7	122
101	700	37.1	56.6	39.8	19.1	40.6	49.4	154
101	800	37.8	57.5	40.8	20.2	41.1	49.2	198

(e) **Accuracy/speed trade-off** RetinaNet (on `test-dev`)

Table 1. **Ablation experiments for RetinaNet and Focal Loss (FL).** All models are trained on `trainval35k` and tested on `minival` unless noted. If not specified, default values are: $\gamma = 2$; anchors for 3 scales and 3 aspect ratios; ResNet-50-FPN backbone; and a 600 pixel train and test image scale. (a) RetinaNet with α-balanced CE achieves at most 31.1 AP. (b) In contrast, using FL with the same exact network gives a 2.9 AP gain and is fairly robust to exact γ/α settings. (c) Using 2-3 scale and 3 aspect ratio anchors yields good results after which point performance saturates. (d) FL outperforms the best variants of online hard example mining (OHEM) [31, 22] by over 3 points AP. (e) Accuracy/Speed trade-off of RetinaNet on `test-dev` for various network depths and image scales (see also Figure 2).

the start of training every anchor should be labeled as foreground with confidence of $\sim\pi$. We use $\pi = .01$ in all experiments, although results are robust to the exact value. As explained in §3.3, this initialization prevents the large number of background anchors from generating a large, destabilizing loss value in the first iteration of training.

Optimization: RetinaNet is trained with stochastic gradient descent (SGD). We use synchronized SGD over 8 GPUs with a total of 16 images per minibatch (2 images per GPU). Unless otherwise specified, all models are trained for 90k iterations with an initial learning rate of 0.01, which is then divided by 10 at 60k and again at 80k iterations. We use horizontal image flipping as the only form of data augmentation unless otherwise noted. Weight decay of 0.0001 and momentum of 0.9 are used. The training loss is the sum the focal loss and the standard smooth L_1 loss used for box regression [10]. Training time ranges between 10 and 35 hours for the models in Table 1e.

5. Experiments

We present experimental results on the bounding box detection track of the challenging COCO benchmark [21]. For training, we follow common practice [1, 20] and use the COCO `trainval35k` split (union of 80k images from `train` and a random 35k subset of images from the 40k image `val` split). We report lesion and sensitivity studies by evaluating on the `minival` split (the remaining 5k images from `val`). For our main results, we report COCO AP on the `test-dev` split, which has no public labels and requires use of the evaluation server.

5.1. Training Dense Detection

We run numerous experiments to analyze the behavior of the loss function for dense detection along with various optimization strategies. For all experiments we use depth 50 or 101 ResNets [16] with a Feature Pyramid Network (FPN) [20] constructed on top. For all ablation studies we use an image scale of 600 pixels for training and testing.

Network Initialization: Our first attempt to train RetinaNet uses standard cross entropy (CE) loss without any modifications to the initialization or learning strategy. This fails quickly, with the network diverging during training. However, simply initializing the last layer of our model such that the prior probability of detecting an object is $\pi = .01$ (see §4.1) enables effective learning. Training RetinaNet with ResNet-50 and this initialization already yields a respectable AP of 30.2 on COCO. Results are insensitive to the exact value of π so we use $\pi = .01$ for all experiments.

α	AP	AP_{50}	AP_{75}
.10	0.0	0.0	0.0
.25	10.8	16.0	11.7
.50	30.2	46.7	32.8
.75	31.1	49.4	33.0
.90	30.8	49.7	32.3
.99	28.7	47.4	29.9
.999	25.1	41.7	26.1

(a) **Varying α for CE loss** ($\gamma = 0$)

γ	α	AP	AP_{50}	AP_{75}
0	.75	31.1	49.4	33.0
0.1	.75	31.4	49.9	33.1
0.2	.75	31.9	50.7	33.4
0.5	.50	32.9	51.7	35.2
1.0	.25	33.7	52.0	36.2
2.0	.25	**34.0**	**52.5**	**36.5**
5.0	.25	32.2	49.6	34.8

(b) **Varying γ for FL** (w. optimal α)

#sc	#ar	AP	AP_{50}	AP_{75}
1	1	30.3	49.0	31.8
2	1	31.9	50.0	34.0
3	1	31.8	49.4	33.7
1	3	32.4	52.3	33.9
2	3	**34.2**	**53.1**	**36.5**
3	3	34.0	52.5	**36.5**
4	3	33.8	52.1	36.2

(c) **Varying anchor scales and aspects**

method	batch size	nms thr	AP	AP_{50}	AP_{75}
OHEM	128	.7	31.1	47.2	33.2
OHEM	256	.7	31.8	48.8	33.9
OHEM	512	.7	30.6	47.0	32.6
OHEM	128	.5	32.8	50.3	35.1
OHEM	256	.5	31.0	47.4	33.0
OHEM	512	.5	27.6	42.0	29.2
OHEM 1:3	128	.5	31.1	47.2	33.2
OHEM 1:3	256	.5	28.3	42.4	30.3
OHEM 1:3	512	.5	24.0	35.5	25.8
FL	n/a	n/a	**36.0**	**54.9**	**38.7**

(d) **FL *vs*. OHEM** baselines (with ResNet-101-FPN)

depth	scale	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	time
50	400	30.5	47.8	32.7	11.2	33.8	46.1	64
50	500	32.5	50.9	34.8	13.9	35.8	46.7	72
50	600	34.3	53.2	36.9	16.2	37.4	47.4	98
50	700	35.1	54.2	37.7	18.0	39.3	46.4	121
50	800	35.7	55.0	38.5	18.9	38.9	46.3	153
101	400	31.9	49.5	34.1	11.6	35.8	48.5	81
101	500	34.4	53.1	36.8	14.7	38.5	49.1	90
101	600	36.0	55.2	38.7	17.4	39.6	49.7	122
101	700	37.1	56.6	39.8	19.1	40.6	49.4	154
101	800	37.8	57.5	40.8	20.2	41.1	49.2	198

(e) **Accuracy/speed trade-off** RetinaNet (on `test-dev`)

表1. RetinaNet与焦点损失（FL）的消融实验。除非特别说明，所有模型均在trainval35k上训练并在minival上测试。若未明确指定，默认参数为：$\gamma = 2$；3种尺度和3种长宽比的锚框；ResNet-50-FPN骨干网络；600像素的训练及测试图像尺度。（a）使用α平衡交叉熵的RetinaNet最高仅获得31.1 AP。（b）相比之下，在相同网络结构中使用FL可获得2.9 AP提升，且对精确的γ/α设置具有较强鲁棒性。（c）采用2-3种尺度和3种长宽比的锚框可获得良好结果，超过此范围后性能趋于饱和。（d）FL以超过3点AP的优势优于在线难例挖掘（OHEM）[31, 22]的最佳变体。（e）不同网络深度与图像尺度下RetinaNet在test-dev集上的精度/速度权衡（另见图2）。

在训练开始时，每个锚点都应标记为前景，置信度为$\sim\pi$。我们在所有实验中均使用$\pi = .01$，尽管具体数值对结果影响不大。如§3.3节所述，这种初始化方式能防止大量背景锚点在训练的首轮迭代中产生巨大且不稳定的损失值。

优化：RetinaNet采用随机梯度下降（SGD）进行训练。我们使用同步SGD，在8个GPU上每个小批次共处理16张图像（每个GPU处理2张图像）。除非另有说明，所有模型均训练90k次迭代，初始学习率为0.01，随后在60k和80k次迭代时分别将学习率除以10。除非特别注明，我们仅使用水平图像翻转作为数据增强方式。权重衰减设为0.0001，动量为0.9。训练损失由焦点损失和用于边界框回归的标准平滑L_1损失相加而成[10]。表1e中列出的模型训练时间在10至35小时之间。

5. 实验

我们在具有挑战性的COCO基准测试[21]的边界框检测任务上展示了实验结果。训练阶段遵循通用实践[1, 20]，采用COCO trainval35k数据集划分（包含来自训练集以及从40k图像验证集中随机抽取的35k子集）。我们通过在minival子集（验证集中剩余的5k图像）上进行评估来报告病灶和敏感性研究。对于我们的主要结果，我们在test-dev子集上报告COCO AP，该子集没有公开标签，需要使用评估服务器。

5.1. 训练密集检测

我们进行了大量实验，以分析密集检测的损失函数行为及多种优化策略。所有实验均采用深度为50或101的ResNet[16]架构，并在其顶部构建特征金字塔网络（FPN）[20]。所有消融研究均使用600像素的图像尺度进行训练和测试。

网络初始化：我们首次尝试训练RetinaNet时使用了标准的交叉熵（CE）损失函数，未对初始化或学习策略进行任何修改。这很快导致失败，网络在训练过程中发散。然而，仅通过初始化模型的最后一层，使检测到目标的先验概率为$\pi = .01$（参见§4.1），即可实现有效学习。使用ResNet-50及此初始化方法训练RetinaNet，在COCO数据集上已能获得30.2 AP的优异结果。实验对π的具体取值不敏感，因此所有实验均采用$\pi = .01$。

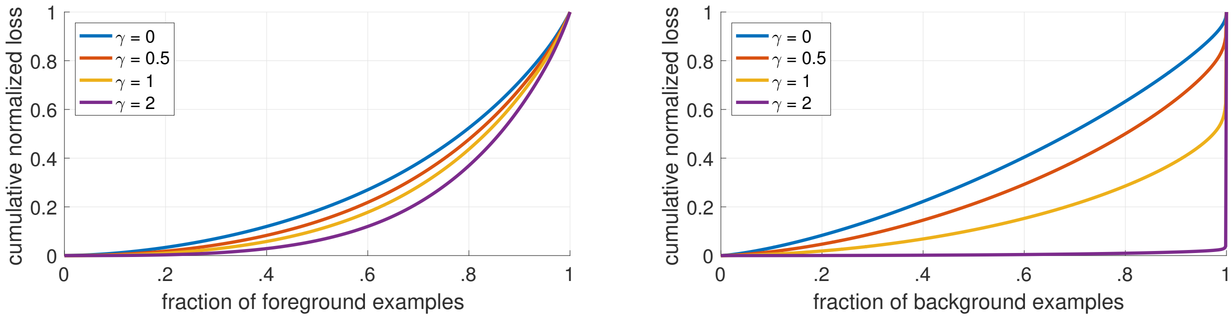


Figure 4. Cumulative distribution functions of the normalized loss for positive and negative samples for different values of γ for a *converged* model. The effect of changing γ on the distribution of the loss for positive examples is minor. For negatives, however, increasing γ heavily concentrates the loss on hard examples, focusing nearly all attention away from easy negatives.

Balanced Cross Entropy: Our next attempt to improve learning involved using the α-balanced CE loss described in §3.1. Results for various α are shown in Table 1a. Setting $\alpha = .75$ gives a gain of 0.9 points AP.

Focal Loss: Results using our proposed focal loss are shown in Table 1b. The focal loss introduces one new hyperparameter, the focusing parameter γ, that controls the strength of the modulating term. When $\gamma = 0$, our loss is equivalent to the CE loss. As γ increases, the shape of the loss changes so that "easy" examples with low loss get further discounted, see Figure 1. FL shows large gains over CE as γ is increased. With $\gamma = 2$, FL *yields a 2.9 AP improvement over the α-balanced CE loss.*

For the experiments in Table 1b, for a fair comparison we find the best α for each γ. We observe that lower α's are selected for higher γ's (as easy negatives are down-weighted, less emphasis needs to be placed on the positives). Overall, however, the benefit of changing γ is much larger, and indeed the best α's ranged in just [.25,.75] (we tested $\alpha \in [.01, .999]$). We use $\gamma = 2.0$ with $\alpha = .25$ for all experiments but $\alpha = .5$ works nearly as well (.4 AP lower).

Analysis of the Focal Loss: To understand the focal loss better, we analyze the empirical distribution of the loss of a *converged* model. For this, we take take our default ResNet-101 600-pixel model trained with $\gamma = 2$ (which has 36.0 AP). We apply this model to a large number of random images and sample the predicted probability for $\sim 10^7$ negative windows and $\sim 10^5$ positive windows. Next, separately for positives and negatives, we compute FL for these samples, and normalize the loss such that it sums to one. Given the normalized loss, we can sort the loss from lowest to highest and plot its cumulative distribution function (CDF) for both positive and negative samples and for different settings for γ (even though model was trained with $\gamma = 2$).

Cumulative distribution functions for positive and negative samples are shown in Figure 4. If we observe the positive samples, we see that the CDF looks fairly similar for different values of γ. For example, approximately 20% of the hardest positive samples account for roughly half of the positive loss, as γ increases more of the loss gets concentrated in the top 20% of examples, but the effect is minor.

The effect of γ on negative samples is dramatically different. For $\gamma = 0$, the positive and negative CDFs are quite similar. However, as γ increases, substantially more weight becomes concentrated on the hard negative examples. In fact, with $\gamma = 2$ (our default setting), the vast majority of the loss comes from a small fraction of samples. As can be seen, FL can effectively discount the effect of easy negatives, focusing all attention on the hard negative examples.

Online Hard Example Mining (OHEM): [31] proposed to improve training of two-stage detectors by constructing minibatches using high-loss examples. Specifically, in OHEM each example is scored by its loss, non-maximum suppression (nms) is then applied, and a minibatch is constructed with the highest-loss examples. The nms threshold and batch size are tunable parameters. Like the focal loss, OHEM puts more emphasis on misclassified examples, but unlike FL, OHEM completely discards easy examples. We also implement a variant of OHEM used in SSD [22]: after applying nms to all examples, the minibatch is constructed to enforce a 1:3 ratio between positives and negatives to help ensure each minibatch has enough positives.

We test both OHEM variants in our setting of one-stage detection which has large class imbalance. Results for the original OHEM strategy and the 'OHEM 1:3' strategy for selected batch sizes and nms thresholds are shown in Table 1d. These results use ResNet-101, our baseline trained with FL achieves 36.0 AP for this setting. In contrast, the best setting for OHEM (no 1:3 ratio, batch size 128, nms of .5) achieves 32.8 AP. This is a gap of 3.2 AP, showing FL is more effective than OHEM for training dense detectors. We note that we tried other parameter setting and variants for OHEM but did not achieve better results.

Hinge Loss: Finally, in early experiments, we attempted to train with the hinge loss [13] on p_t, which sets loss to 0 above a certain value of p_t. However, this was unstable and we did not manage to obtain meaningful results. Results exploring alternate loss functions are in the appendix.

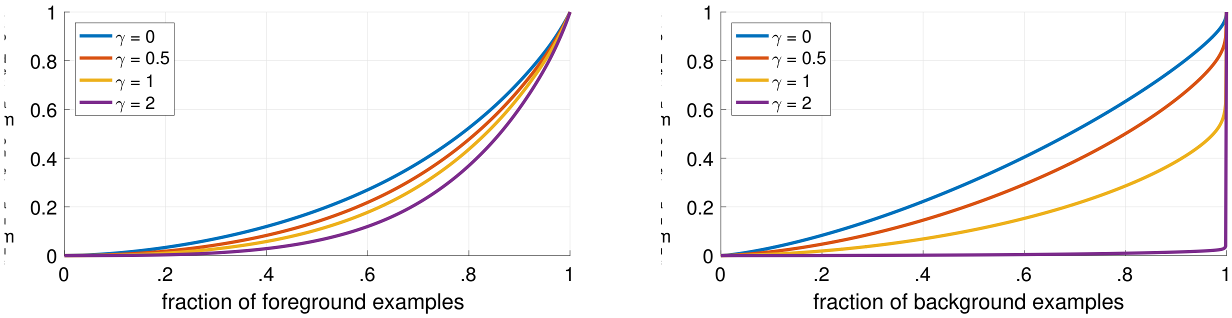


图4. 针对不同γ值，*converged*模型正负样本归一化损失的累积分布函数。改变γ对正样本损失分布的影响较小。然而对于负样本，增加γ会使损失高度集中于困难样本，几乎将所有注意力从简单负样本上转移开。

平衡交叉熵：我们改进学习的下一个尝试是使用在§3.1节中描述的α平衡CE损失。不同α的结果如表1a所示。设定$\alpha = .75$可使AP提升0.9个点。

Focal Loss：使用我们提出的focal loss的结果如表1b所示。Focal loss引入了一个新的超参数——聚焦参数γ，它控制调制项的强度。当$\gamma =$为0时，我们的损失函数等价于CE损失。随着γ的增加，损失函数的形状发生变化，使得低损失的“简单”样本进一步被折扣，如图1所示。随着γ的增加，FL相比CE显示出显著的提升。当$\gamma =$为2时，FL *yields a 2.9 AP improvement over the α-balanced CE loss*。

对于表1b中的实验，为了公平比较，我们为每个γ找到了最佳的α。我们观察到，较高的γ对应选择了较低的α（因为容易的负样本被降权，所以无需过度关注正样本）。然而总体而言，调整γ带来的收益要大得多，实际上最佳α的范围仅为[0.25, 0.75]（我们测试了$\alpha \in [.01, .999]$）。在所有实验中我们使用$\gamma = 2.0$并设定$\alpha = .25$，但$\alpha = .5$的效果也几乎相当（AP值低0.4）。

焦点损失分析：为了更好地理解焦点损失，我们分析了*converged*模型损失的实证分布。为此，我们采用默认的ResNet-101 600像素模型，该模型使用$\gamma = 2$进行训练（AP为36.0）。我们将此模型应用于大量随机图像，并对~10^7个负样本窗口和~10^5个正样本窗口的预测概率进行采样。接着，分别针对正负样本计算这些样本的焦点损失，并对损失进行归一化处理，使其总和为1。基于归一化后的损失，我们可以将损失从低到高排序，并绘制正负样本在不同γ (设置下的累积分布函数（CDF），尽管模型是用$\gamma = 2$)训练的。

正负样本的累积分布函数如图4所示。如果我们观察正样本，会发现不同{v*}值下的CDF看起来相当相似。例如，约20%的最难正样本占据了大约一半的正损失，随着γ增加，更多的损失集中在排名前20%的样本中，但影响较小。

γ对负样本的影响截然不同。当$\gamma =$为0时，正负样本的累积分布函数相当接近。但随着γ的增加，损失权重明显向困难负样本集中。实际上，在$\gamma =$设为2时（我们的默认设置），绝大部分损失仅来自极小比例的样本。可见，Focal Loss能有效削弱简单负样本的影响，将所有注意力集中在困难负样本上。

在线难例挖掘（OHEM）：[31]提出通过使用高损失样本构建小批量数据来改进两阶段检测器的训练。具体而言，在OHEM中，每个样本根据其损失进行评分，随后应用非极大值抑制（nms），并利用损失最高的样本构建小批量数据。nms阈值和批量大小是可调参数。与焦点损失类似，OHEM更关注误分类样本，但不同于FL的是，OHEM完全舍弃了简单样本。我们还实现了SSD[22]中使用的一种OHEM变体：在对所有样本应用nms后，构建小批量数据时强制保持正负样本1:3的比例，以确保每个小批量包含足够的正样本。

我们在单阶段检测（存在严重类别不平衡）的设置中测试了两种OHEM变体。原始OHEM策略与“OHEM 1:3”策略在选定批次大小及非极大值抑制阈值下的结果如表1d所示。这些实验使用ResNet-101，我们采用FL训练的基线模型在此设置下达到36.0 AP。相比之下，OHEM的最佳设置（未采用1:3比例，批次大小128，非极大值抑制阈值0.5）仅获得32.8 AP。两者存在3.2 AP的差距，表明在训练密集检测器时FL比OHEM更有效。需要说明的是，我们尝试了OHEM的其他参数设置与变体，但均未获得更好结果。

铰链损失：在早期实验中，我们曾尝试在p_t上使用铰链损失[13]进行训练，该损失在p_t超过特定值时将损失设为0。然而，这种方法不稳定，我们未能获得有意义的结果。关于其他损失函数的探索结果详见附录。

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [16]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [20]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [17]	Inception-ResNet-v2 [34]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [32]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [27]	DarkNet-19 [27]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [22, 9]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [9]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet (ours)	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet (ours)	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2

Table 2. **Object detection** *single-model* results (bounding box AP), *vs*. state-of-the-art on COCO `test-dev`. We show results for our RetinaNet-101-800 model, trained with scale jitter and for 1.5× longer than the same model from Table 1e. Our model achieves top results, outperforming both one-stage and two-stage models. For a detailed breakdown of speed versus accuracy see Table 1e and Figure 2.

5.2. Model Architecture Design

Anchor Density: One of the most important design factors in a one-stage detection system is how densely it covers the space of possible image boxes. Two-stage detectors can classify boxes at any position, scale, and aspect ratio using a region pooling operation [10]. In contrast, as one-stage detectors use a fixed sampling grid, a popular approach for achieving high coverage of boxes in these approaches is to use multiple 'anchors' [28] at each spatial position to cover boxes of various scales and aspect ratios.

We sweep over the number of scale and aspect ratio anchors used at each spatial position and each pyramid level in FPN. We consider cases from a single square anchor at each location to 12 anchors per location spanning 4 sub-octave scales ($2^{k/4}$, for $k \leq 3$) and 3 aspect ratios [0.5, 1, 2]. Results using ResNet-50 are shown in Table 1c. A surprisingly good AP (30.3) is achieved using just one square anchor. However, the AP can be improved by nearly 4 points (to 34.0) when using 3 scales and 3 aspect ratios per location. We used this setting for all other experiments in this work.

Finally, we note that increasing beyond 6-9 anchors did not shown further gains. Thus while two-stage systems can classify arbitrary boxes in an image, the saturation of performance w.r.t. density implies the higher potential density of two-stage systems may not offer an advantage.

Speed versus Accuracy: Larger backbone networks yield higher accuracy, but also slower inference speeds. Likewise for input image scale (defined by the shorter image side). We show the impact of these two factors in Table 1e. In Figure 2 we plot the speed/accuracy trade-off curve for RetinaNet and compare it to recent methods using public numbers on COCO `test-dev`. The plot reveals that RetinaNet, enabled by our focal loss, forms an upper envelope over all existing methods, discounting the low-accuracy regime. RetinaNet with ResNet-101-FPN and a 600 pixel image scale (which we denote by RetinaNet-101-600 for simplicity) matches the accuracy of the recently published ResNet-101-FPN Faster R-CNN [20], while running in 122 ms per image compared to 172 ms (both measured on an Nvidia M40 GPU). Using larger scales allows RetinaNet to surpass the accuracy of all two-stage approaches, while still being faster. For faster runtimes, there is only one operating point (500 pixel input) at which using ResNet-50-FPN improves over ResNet-101-FPN. Addressing the high frame rate regime will likely require special network design, as in [27], and is beyond the scope of this work. We note that after publication, faster and more accurate results can now be obtained by a variant of Faster R-CNN from [12].

5.3. Comparison to State of the Art

We evaluate RetinaNet on the challenging COCO dataset and compare `test-dev` results to recent state-of-the-art methods including both one-stage and two-stage models. Results are presented in Table 2 for our RetinaNet-101-800 model trained using scale jitter and for 1.5× longer than the models in Table 1e (giving a 1.3 AP gain). Compared to existing one-stage methods, our approach achieves a healthy 5.9 point AP gap (39.1 *vs*. 33.2) with the closest competitor, DSSD [9], while also being faster, see Figure 2. Compared to recent two-stage methods, RetinaNet achieves a 2.3 point gap above the top-performing Faster R-CNN model based on Inception-ResNet-v2-TDM [32]. Plugging in ResNeXt-32x8d-101-FPN [38] as the RetinaNet backbone further improves results another 1.7 AP, surpassing 40 AP on COCO.

6. Conclusion

In this work, we identify class imbalance as the primary obstacle preventing one-stage object detectors from surpassing top-performing, two-stage methods. To address this, we propose the *focal loss* which applies a modulating term to the cross entropy loss in order to focus learning on hard negative examples. Our approach is simple and highly effective. We demonstrate its efficacy by designing a fully convolutional one-stage detector and report extensive experimental analysis showing that it achieves state-of-the-art accuracy and speed. Source code is available at https://github.com/facebookresearch/Detectron [12].

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [16]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [20]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [17]	Inception-ResNet-v2 [34]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [32]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [27]	DarkNet-19 [27]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [22, 9]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [9]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet (ours)	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet (ours)	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2

表2. 目标检测 *single-model* 结果（边界框AP），*vs*。在COCO test-dev数据集上的最先进水平。我们展示了使用尺度抖动训练、且训练时长比表1e中相同模型长1.5×的RetinaNet-101-800模型的结果。我们的模型取得了最佳结果，优于单阶段和双阶段模型。关于速度与准确性的详细分析，请参见表1e和图2。

5.2. 模型架构设计

锚点密度：单阶段检测系统中最重要的设计因素之一是其对可能图像框空间的覆盖密度。两阶段检测器可以通过区域池化操作[10]对任意位置、尺度和宽高比的框进行分类。相比之下，由于单阶段检测器使用固定的采样网格，这些方法中实现高框覆盖率的一种常用策略是在每个空间位置使用多个“锚点”[28]来覆盖不同尺度和宽高比的框。

我们在FPN的每个空间位置和金字塔层级上，对使用的尺度和宽高比锚框数量进行了全面扫描。我们考虑了从每个位置仅使用单个正方形锚框，到每个位置使用12个锚框的情况——这些锚框覆盖4个亚八度尺度（2^$^{k/4}$，对于^$k \le 3$）和3种宽高比[0.5, 1, 2]。使用ResNet-50的结果如表1c所示。仅使用单个正方形锚框即可达到惊人的30.3 AP。然而，当每个位置使用3种尺度和3种宽高比时，AP可提升近4个点（至34.0）。本工作中所有其他实验均采用此设置。

最后，我们注意到，将锚点数量增加到6-9个以上并未带来进一步的性能提升。因此，尽管两阶段系统能够对图像中的任意框进行分类，但性能随密度增加而饱和的现象意味着，两阶段系统潜在的高密度优势可能无法转化为实际收益。

速度与准确性的权衡：更大的骨干网络带来更高的准确性，但也会降低推理速度。输入图像尺度（由图像短边定义）同样如此。我们在表1e中展示了这两个因素的影响。图2绘制了RetinaNet的速度/准确性权衡曲线，并与近期方法在COCO test-dev数据集上的公开数据进行了对比。图表显示，得益于我们的焦点损失函数，RetinaNet在所有现有方法之上形成了性能上限包络线（低精度区域除外）。采用ResNet-101-FPN骨干网络和600像素图像尺度的RetinaNet（为简洁记为RetinaNet-101-600）达到了近期发表的ResNet-101-FPN Faster R-CNN[20]的精度水平，同时每张图像处理时间仅需122毫秒与172毫秒相比（两者均在Nvidia M40 GPU上测得）。使用更大的尺度使RetinaNet能够超越所有两阶段方法的精度，同时仍保持更快的速度。对于更短的运行时间，仅有一个操作点（500像素输入）在使用ResNet-50-FPN时优于ResNet-101-FPN。要应对高帧率场景，可能需要如[27]中所述的特殊网络设计，这超出了本工作的范围。我们注意到，在本文发表后，通过[12]中Faster R-CNN的变体现已能获得更快、更准确的结果。

5.3. 与现有技术的比较

我们在具有挑战性的COCO数据集上评估RetinaNet，并将测试开发集结果与包括单阶段和双阶段模型在内的近期先进方法进行比较。表2展示了我们使用尺度抖动训练、训练时长比表1e中模型长1.5×倍的RetinaNet-101-800模型的结果（带来1.3 AP提升）。与现有单阶段方法相比，我们的方法以5.9个AP点的显著优势（39.1 *vs*对比33.2）超越最接近的竞争者DSSD[9]，同时速度更快，详见图2。与近期双阶段方法相比，RetinaNet比基于Inception-ResNet-v2-TDM[32]的最高性能Faster R-CNN模型高出2.3个AP点。采用ResNeXt-32x8d-101-FPN[38]作为RetinaNet骨干网络可进一步将结果提升1.7 AP，在COCO数据集上突破40 AP大关。

6. 结论

在本研究中，我们发现类别不平衡是阻碍单阶段目标检测器超越顶级两阶段方法的主要障碍。为解决这一问题，我们提出了*focal loss*方法，该方法通过向交叉熵损失引入调制项，使模型聚焦于难以分类的负样本学习。我们的方法简洁高效，通过设计全卷积单阶段检测器验证了其优越性，大量实验分析表明该方法在精度与速度上均达到业界最优水平。源代码发布于https://github.com/facebookresearch/Detectron [12]。

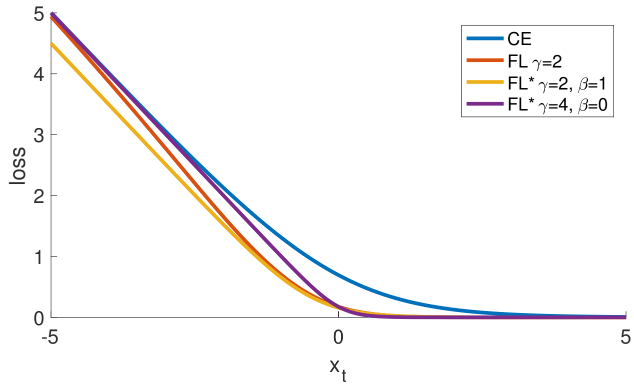


Figure 5. Focal loss variants compared to the cross entropy as a function of $x_t = yx$. Both the original FL and alternate variant FL* reduce the relative loss for well-classified examples ($x_t > 0$).

loss	γ	β	AP	AP$_{50}$	AP$_{75}$
CE	–	–	31.1	49.4	33.0
FL	2.0	–	34.0	52.5	36.5
FL*	2.0	1.0	33.8	52.7	36.3
FL*	4.0	0.0	33.9	51.8	36.4

Table 3. Results of FL and FL* versus CE for select settings.

Appendix A: Focal Loss*

The exact form of the focal loss is not crucial. We now show an alternate instantiation of the focal loss that has similar properties and yields comparable results. The following also gives more insights into properties of the focal loss.

We begin by considering both cross entropy (CE) and the focal loss (FL) in a slightly different form than in the main text. Specifically, we define a quantity x_t as follows:

$$x_t = yx, \tag{6}$$

where $y \in \{\pm 1\}$ specifies the ground-truth class as before. We can then write $p_t = \sigma(x_t)$ (this is compatible with the definition of p_t in Equation 2). An example is correctly classified when $x_t > 0$, in which case $p_t > .5$.

We can now define an alternate form of the focal loss in terms of x_t. We define p_t^* and FL* as follows:

$$p_t^* = \sigma(\gamma x_t + \beta), \tag{7}$$

$$\text{FL}^* = -\log(p_t^*)/\gamma. \tag{8}$$

FL* has two parameters, γ and β, that control the steepness and shift of the loss curve. We plot FL* for two selected settings of γ and β in Figure 5 alongside CE and FL. As can be seen, like FL, FL* with the selected parameters diminishes the loss assigned to well-classified examples.

We trained RetinaNet-50-600 using identical settings as before but we swap out FL for FL* with the selected parameters. These models achieve nearly the same AP as those trained with FL, see Table 3. In other words, FL* is a reasonable alternative for the FL that works well in practice.

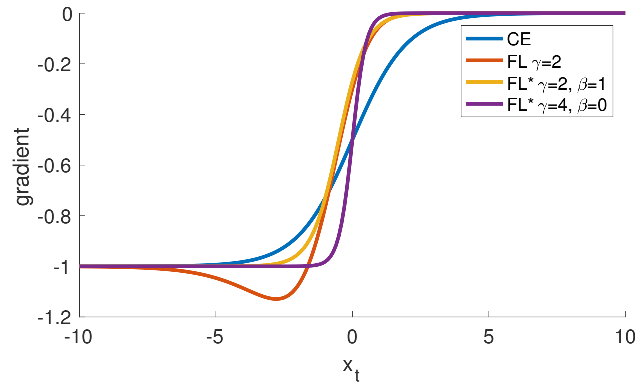


Figure 6. Derivates of the loss functions from Figure 5 w.r.t. x.

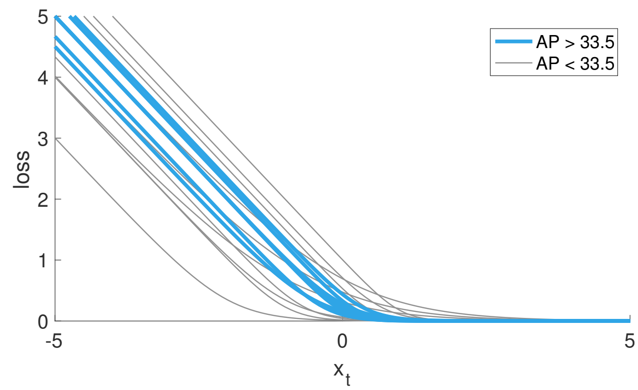


Figure 7. Effectiveness of FL* with various settings γ and β. The plots are color coded such that effective settings are shown in blue.

We found that various γ and β settings gave good results. In Figure 7 we show results for RetinaNet-50-600 with FL* for a wide set of parameters. The loss plots are color coded such that effective settings (models converged and with AP over 33.5) are shown in blue. We used $\alpha = .25$ in all experiments for simplicity. As can be seen, losses that reduce weights of well-classified examples ($x_t > 0$) are effective.

More generally, we expect any loss function with similar properties as FL or FL* to be equally effective.

Appendix B: Derivatives

For reference, derivates for CE, FL, and FL* w.r.t. x are:

$$\frac{d\text{CE}}{dx} = y(p_t - 1) \tag{9}$$

$$\frac{d\text{FL}}{dx} = y(1 - p_t)^\gamma(\gamma p_t \log(p_t) + p_t - 1) \tag{10}$$

$$\frac{d\text{FL}^*}{dx} = y(p_t^* - 1) \tag{11}$$

Plots for selected settings are shown in Figure 6. For all loss functions, the derivative tends to -1 or 0 for high-confidence predictions. However, unlike CE, for effective settings of both FL and FL*, the derivative is small as soon as $x_t > 0$.

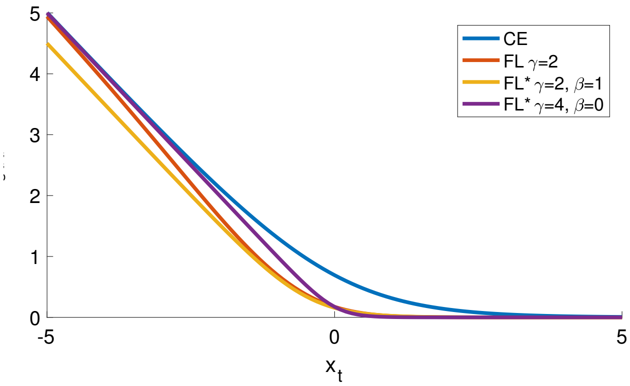


图5. 作为$x_t = yx$函数的焦点损失变体与交叉熵对比。原始FL及其变体FL*均降低了分类良好样本($x_t > 0$)的相对损失。

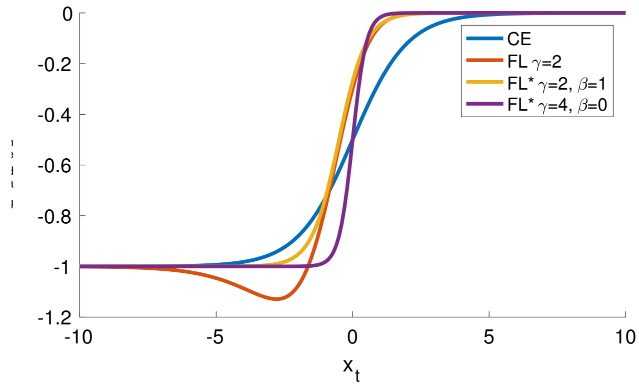


图6. 图5中损失函数关于x的导数。

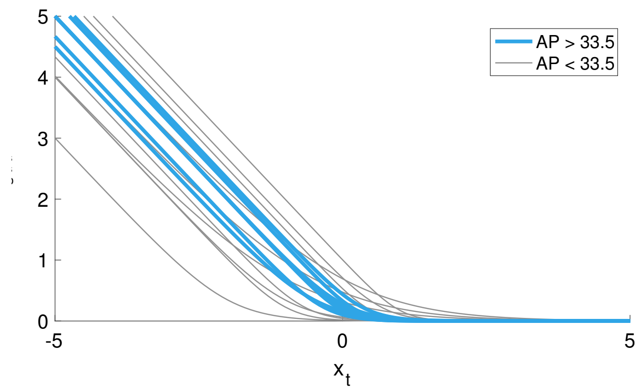


图7. FL*在不同设置γ和β下的有效性。图表采用颜色编码，有效设置以蓝色显示。

loss	γ	β	AP	AP$_{50}$	AP$_{75}$
CE	–	–	31.1	49.4	33.0
FL	2.0	–	34.0	52.5	36.5
FL*	2.0	1.0	33.8	52.7	36.3
FL*	4.0	0.0	33.9	51.8	36.4

表3. 在选定设置下FL与FL*对比CE的结果。

附录A：焦点损失*

焦点损失的确切形式并不关键。我们现在展示焦点损失的另一种实例化，它具有相似的性质并能产生可比较的结果。以下内容也提供了对焦点损失性质的更多见解。

我们首先以与正文略有不同的形式考虑交叉熵（CE）和焦点损失（FL）。具体来说，我们定义量 x_t 如下：

$$x_t = yx, \tag{6}$$

其中 $y \in \{\pm 1\}$ 如前所述指定了真实类别。我们可以将其写作 $p_t = \sigma(x_t)$ (这与公式2中 p_t 的定义相符)。当 $x_t > 0$ 时示例被正确分类，此时 $p_t > .5$。

我们现在可以根据x_t定义焦点损失的另一种形式。我们定义p_t^*和FL*如下：

$$p_t^* = \sigma(\gamma x_t + \beta), \tag{7}$$

$$\text{FL}^* = -\log(p_t^*)/\gamma. \tag{8}$$

FL*有两个参数，γ和β，用于控制损失曲线的陡峭程度和平移。我们在图5中绘制了针对γ和β两组选定设置的FL*，并与CE和FL并列展示。如图所示，与FL类似，采用选定参数的FL*减少了对已正确分类样本分配的损失。

我们使用与之前相同的设置训练了RetinaNet-50-600，但将FL替换为选定了参数的FL*。这些模型达到了与使用FL训练模型几乎相同的AP，详见表3。换言之，FL*是FL在实际应用中表现良好的合理替代方案。

我们发现不同的γ和β设置都能取得良好效果。在图7中，我们展示了RetinaNet-50-600搭配FL*在广泛参数范围内的实验结果。损失曲线采用颜色编码，其中有效设置（模型收敛且AP超过33.5）以蓝色标示。为简化操作，所有实验均采用$\alpha = .25$。如图所示，能降低正确分类样本权重的损失函数（$x_t > 0$）具有显著效果。

更一般地说，我们期望任何具有与FL或FL{v*}类似性质的损失函数都能同样有效。

附录B：导数

作为参考，CE、FL和FL*关于x的导数分别为：

$$\frac{d\text{CE}}{dx} = y(p_t - 1) \tag{9}$$

$$\frac{d\text{FL}}{dx} = y(1 - p_t)^\gamma(\gamma p_t \log(p_t) + p_t - 1) \tag{10}$$

$$\frac{d\text{FL}^*}{dx} = y(p_t^* - 1) \tag{11}$$

选定设置的绘图如图6所示。对于所有损失函数，高置信度预测的导数倾向于-1或0。然而，与交叉熵不同，在FL和FL*的有效设置下，只要$x_t > 0$，导数就会很小。

References

[1] S. Bell, C. L. Zitnick, K. Bala, and R. Girshick. Inside-outside net: Detecting objects in context with skip pooling and recurrent neural networks. In *CVPR*, 2016. 6

[2] S. R. Bulo, G. Neuhold, and P. Kontschieder. Loss max-pooling for semantic image segmentation. In *CVPR*, 2017. 3

[3] J. Dai, Y. Li, K. He, and J. Sun. R-FCN: Object detection via region-based fully convolutional networks. In *NIPS*, 2016. 1

[4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *CVPR*, 2005. 2

[5] P. Dollár, Z. Tu, P. Perona, and S. Belongie. Integral channel features. In *BMVC*, 2009. 2, 3

[6] D. Erhan, C. Szegedy, A. Toshev, and D. Anguelov. Scalable object detection using deep neural networks. In *CVPR*, 2014. 2

[7] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes (VOC) Challenge. *IJCV*, 2010. 2

[8] P. F. Felzenszwalb, R. B. Girshick, and D. McAllester. Cascade object detection with deformable part models. In *CVPR*, 2010. 2, 3

[9] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, and A. C. Berg. DSSD: Deconvolutional single shot detector. *arXiv:1701.06659*, 2016. 1, 2, 8

[10] R. Girshick. Fast R-CNN. In *ICCV*, 2015. 1, 2, 4, 6, 8

[11] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014. 1, 2, 5

[12] R. Girshick, I. Radosavovic, G. Gkioxari, P. Dollár, and K. He. Detectron. `https://github.com/facebookresearch/detectron`, 2018. 8

[13] T. Hastie, R. Tibshirani, and J. Friedman. *The elements of statistical learning*. Springer series in statistics Springer, Berlin, 2008. 3, 7

[14] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. In *ICCV*, 2017. 1, 2, 4

[15] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *ECCV*. 2014. 2

[16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 2, 4, 5, 6, 8

[17] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, and K. Murphy. Speed/accuracy trade-offs for modern convolutional object detectors. In *CVPR*, 2017. 2, 8

[18] A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. In *NIPS*, 2012. 2

[19] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989. 2

[20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017. 1, 2, 4, 5, 6, 8

[21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*, 2014. 1, 6

[22] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, and S. Reed. SSD: Single shot multibox detector. In *ECCV*, 2016. 1, 2, 3, 6, 7, 8

[23] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015. 4

[24] P. O. Pinheiro, R. Collobert, and P. Dollar. Learning to segment object candidates. In *NIPS*, 2015. 2, 4

[25] P. O. Pinheiro, T.-Y. Lin, R. Collobert, and P. Dollár. Learning to refine object segments. In *ECCV*, 2016. 2

[26] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016. 1, 2

[27] J. Redmon and A. Farhadi. YOLO9000: Better, faster, stronger. In *CVPR*, 2017. 1, 2, 8

[28] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015. 1, 2, 4, 5, 8

[29] H. Rowley, S. Baluja, and T. Kanade. Human face detection in visual scenes. Technical Report CMU-CS-95-158R, Carnegie Mellon University, 1995. 2

[30] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014. 2

[31] A. Shrivastava, A. Gupta, and R. Girshick. Training region-based object detectors with online hard example mining. In *CVPR*, 2016. 2, 3, 6, 7

[32] A. Shrivastava, R. Sukthankar, J. Malik, and A. Gupta. Beyond skip connections: Top-down modulation for object detection. *arXiv:1612.06851*, 2016. 2, 8

[33] K.-K. Sung and T. Poggio. Learning and Example Selection for Object and Pattern Detection. In *MIT A.I. Memo No. 1521*, 1994. 2, 3

[34] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In *AAAI Conference on Artificial Intelligence*, 2017. 8

[35] J. R. Uijlings, K. E. van de Sande, T. Gevers, and A. W. Smeulders. Selective search for object recognition. *IJCV*, 2013. 2, 4

[36] R. Vaillant, C. Monrocq, and Y. LeCun. Original approach for the localisation of objects in images. *IEE Proc. on Vision, Image, and Signal Processing*, 1994. 2

[37] P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *CVPR*, 2001. 2, 3

[38] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017. 8

[39] C. L. Zitnick and P. Dollár. Edge boxes: Locating object proposals from edges. In *ECCV*, 2014. 2

参考文献

[1] S. Bell, C. L. Zitnick, K. Bala, 和 R. Girshick。Inside-Outside Net：通过跳跃池化和循环神经网络在上下文中检测目标。发表于 *CVPR*, 2016. 6[2] S. R. Bulo, G. Neuhold, 和 P. Kontschieder。用于语义图像分割的损失最大池化。发表于 *CVPR*, 2017. 3[3] J. Dai, Y. Li, K. He, 和 J. Sun。R-FCN：通过基于区域的全卷积网络进行目标检测。发表于 *NIPS*, 2016. 1[4] N. Dalal 和 B. Triggs。用于人体检测的定向梯度直方图。发表于 *CVPR*, 2005. 2[5] P. Dollár, Z. Tu, P. Perona, 和 S. Belongie。积分通道特征。发表于 *BMVC*, 2009. 2, 3[6] D. Erhan, C. Szegedy, A. Toshev, 和 D. Anguelov。使用深度神经网络的可扩展目标检测。发表于 *CVPR*, 2014. 2[7] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, 和 A. Zisserman。PASCAL 视觉目标类别 (VOC) 挑战赛。*IJCV*, 2010. 2[8] P. F. Felzenszwalb, R. B. Girshick, 和 D. McAllester。使用可变形部件模型的级联目标检测。发表于 *CVPR*, 2010. 2, 3[9] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, 和 A. C. Berg。DSSD：反卷积单次检测器。*arXiv:1701.06659*, 2016. 1, 2, 8[10] R. Girshick。Fast R-CNN。发表于 *ICCV*, 2015. 1, 2, 4, 6, 8[11] R. Girshick, J. Donahue, T. Darrell, 和 J. Malik。用于精确目标检测和语义分割的丰富特征层次结构。发表于 *CVPR*, 2014. 1, 2, 5[12] R. Girshick, I. Radosavovic, G. Gkioxari, P. Dollár, 和 K. He。Detectron。https://github.com/facebookresearch/detectron, 2018. 8[13] T. Hastie, R. Tibshirani, 和 J. Friedman。*The elements of statistical learning*。Springer 统计学系列，Springer, Berlin, 2008. 3, 7[14] K. He, G. Gkioxari, P. Dollár, 和 R. Girshick。Mask R-CNN。发表于 *ICCV*, 2017. 1, 2, 4[15] K. He, X. Zhang, S. Ren, 和 J. Sun。深度卷积网络中用于视觉识别的空间金字塔池化。发表于 *ECCV*. 2014. 2[16] K. He, X. Zhang, S. Ren, 和 J. Sun。用于图像识别的深度残差学习。发表于 *CVPR*, 2016. 2, 4, 5, 6, 8[17] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, 和 K. Murphy。现代卷积目标检测器的速度/精度权衡。发表于 *CVPR*, 2017. 2, 8[18] A. Krizhevsky, I. Sutskever, 和 G. Hinton。使用深度卷积神经网络的 ImageNet 分类。发表于 *NIPS*, 2012. 2[19] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, 和 L. D. Jackel。反向传播应用于手写邮政编码识别。*Neural compu- tation*, 1989. 2[20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, 和 S. Belongie。用于目标检测的特征金字塔网络。发表于 *CVPR*, 2017. 1, 2, 4, 5, 6, 8

[21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick. Microsoft COCO：上下文中的常见物体。发表于*ECCV*，2014年。1, 6[22] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed. SSD：单次多框检测器。发表于*ECCV*，2016年。1, 2, 3, 6, 7, 8[23] J. Long, E. Shelhamer, T. Darrell. 用于语义分割的全卷积网络。发表于*CVPR*，2015年。4[24] P. O. Pinheiro, R. Collobert, P. Dollar. 学习分割候选物体。发表于*NIPS*，2015年。2, 4[25] P. O. Pinheiro, T.-Y. Lin, R. Collobert, P. Dollár. 学习优化物体分割。发表于*ECCV*，2016年。2[26] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. YOLO：只需看一次的统一实时物体检测。发表于*CVPR*，2016年。1, 2[27] J. Redmon, A. Farhadi. YOLO9000：更好、更快、更强。发表于*CVPR*，2017年。1, 2, 8[28] S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN：利用区域提议网络实现实时物体检测。发表于*NIPS*，2015年。1, 2, 4, 5, 8[29] H. Rowley, S. Baluja, T. Kanade. 视觉场景中的人脸检测。技术报告 CMU-CS-95-158R，卡内基梅隆大学，1995年。2[30] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, Y. LeCun. Overfeat：使用卷积网络进行集成识别、定位与检测。发表于*ICLR*，2014年。2[31] A. Shrivastava, A. Gupta, R. Girshick. 使用在线难例挖掘训练基于区域的物体检测器。发表于*CVPR*，2016年。2, 3, 6, 7[32] A. Shrivastava, R. Sukthankar, J. Malik, A. Gupta. 超越跳跃连接：用于物体检测的自顶向下调制。*arXiv:1612.06851*，2016年。2, 8[33] K.-K. Sung, T. Poggio. 物体与模式检测的学习与示例选择。发表于*MIT A.I. Memo No. 1521*，1994年。2, 3[34] C. Szegedy, S. Ioffe, V. Vanhoucke, A. A. Alemi. Inception-v4, Inception-ResNet 以及残差连接对学习的影响。发表于*AAAI Conference on ArtificialIntelligence*，2017年。8[35] J. R. Uijlings, K. E. van de Sande, T. Gevers, A. W. Smeulders. 用于物体识别的选择性搜索。*IJCV*，2013年。2, 4[36] R. Vaillant, C. Monrocq, Y. LeCun. 图像中物体定位的原创方法。*IEE Proc. on Vision,Image, and Signal Processing*，1994年。2[37] P. Viola, M. Jones. 使用增强级联简单特征的快速物体检测。发表于*CVPR*，2001年。2, 3[38] S. Xie, R. Girshick, P. Dollár, Z. Tu, K. He. 用于深度神经网络的聚合残差变换。发表于*CVPR*，2017年。8[39] C. L. Zitnick, P. Dollár. Edge Boxes：从边缘定位物体提议。发表于*ECCV*，2014年。2